

1 - Modelos supervisados con scikit-learn: entrenamiento básico

Ejercicio práctico para entrenar modelos supervisados básicos usando scikit-learn en Python. El alumno implementará un pipeline completo desde la división de datos, preprocesado, entrenamiento y evaluación con regresión lineal y regresión logística, aplicando validación cruzada y métricas básicas.

Unidad: Validación: train/test, k-fold y selección de hiperparámetros

En esta unidad, pondremos en práctica la **validación cruzada** para evaluar modelos de Machine Learning, implementaremos **pipelines** para asegurar reproducibilidad y automatizaremos la **selección de hiperparámetros** usando `GridSearchCV` y `RandomizedSearchCV` de scikit-learn.

Conceptos clave

- **Validación cruzada k-fold:** Divide el dataset en k partes, entrena y valida el modelo k veces, cada vez con una parte diferente como conjunto de validación.
 - **Pipelines:** Encadenan pasos de preprocesamiento y modelado para evitar fugas de datos y facilitar la experimentación.
 - **GridSearchCV y RandomizedSearchCV:** Métodos para buscar automáticamente los mejores hiperparámetros evaluando combinaciones mediante validación cruzada.
-

Ejercicio práctico

Implementarás un pipeline que incluya preprocesamiento y un modelo de clasificación. Luego, aplicarás validación cruzada para evaluar el rendimiento y usarás `GridSearchCV` y `RandomizedSearchCV` para encontrar los mejores hiperparámetros.

Te dejamos el código para que te guíes en la práctica:

Objetivo general de la práctica

En esta práctica trabajaremos con un problema realista de **clasificación supervisada**, cuyo objetivo es **predecir el churn (abandono) de clientes** en un contexto de e-commerce.

El foco principal **no está en mejorar el modelo a toda costa**, sino en **aprender a evaluar correctamente un modelo de Machine Learning**, evitando errores comunes como:

- sobreajuste (overfitting)
- fuga de datos (data leakage)
- evaluación optimista no realista
- selección incorrecta de hiperparámetros

Para ello, aplicaremos distintas estrategias de validación y técnicas de automatización provistas por *scikit-learn*.

Contexto del problema de negocio

Una empresa de e-commerce quiere anticiparse a la pérdida de clientes.

A partir de información como:

- edad
- región
- comportamiento de compra
- recencia
- gasto total
- engagement con promociones

se busca construir un modelo que permita estimar la probabilidad de que un cliente **abandone la plataforma**.

Este tipo de problema es típico en:

- marketing analytics
- retención de clientes
- modelos de churn
- customer lifetime value

y suele requerir **modelos interpretables**, métricas adecuadas y evaluación rigurosa.

◆ 1. Separación de variables (X / y)

Antes de entrenar cualquier modelo es fundamental distinguir:

- **variables predictoras (X)**
- **variable objetivo (y)**

En este caso:

- **y** representa si el cliente abandonó o no (churn)
- **X** contiene todas las variables demográficas y de comportamiento

Además, se identifican:

- **columnas numéricas** → requieren escalado
- **columnas categóricas** → requieren codificación

Esta separación es clave para poder construir pipelines automáticos y reproducibles.

◆ 2. Preprocesamiento mediante pipelines

En Machine Learning real, los datos **raramente están listos para entrenar**.

Por eso se trabaja con dos tipos de transformaciones:

Variables numéricas

- imputación de valores faltantes (mediana)
- estandarización (StandardScaler)

Variables categóricas

- imputación del valor más frecuente
- transformación a variables binarias (One-Hot Encoding)

Estas transformaciones **no deben aplicarse manualmente fuera del modelo**, ya que eso provocaría **fuga de datos**.

Por este motivo utilizamos:

- Pipeline
- ColumnTransformer

De esta forma, el preprocesamiento se ejecuta **solo con los datos de entrenamiento en cada validación**, manteniendo la coherencia del proceso.

◆ 3. Pipelines: por qué son fundamentales

Un pipeline permite encadenar:

preprocesamiento → modelo

en una única estructura.

Esto garantiza que:

- el escalado no vea datos de test
- la imputación no use información futura
- la validación cruzada sea correcta
- los hiperparámetros se ajusten sin sesgos

En escenarios reales y productivos, **todo modelo debería entrenarse siempre dentro de un pipeline**.

◆ 4. Validación Hold-Out (train / test)

La primera forma de evaluación es la **validación hold-out**, que consiste en:

- dividir el dataset en:
- conjunto de entrenamiento

- conjunto de test
- entrenar únicamente con el set de entrenamiento
- evaluar el desempeño final sobre el set de test

En esta práctica se utiliza:

- división estratificada
- proporción típica (por ejemplo 75% / 25%)

Esta técnica es simple y rápida, pero **depende mucho de una sola partición**, lo que puede generar resultados inestables.

Por eso se complementa con validación cruzada.

◆ 5. Métricas de evaluación

Dado que el problema es de **clasificación binaria**, se utilizan las siguientes métricas:

- **Accuracy**: proporción total de aciertos
- **Precision**: qué tan confiables son los positivos predichos
- **Recall**: qué porcentaje de churn reales se detectan
- **F1-score**: equilibrio entre precision y recall

En problemas de churn, el **F1-score suele ser más representativo** que el accuracy, especialmente cuando las clases están desbalanceadas.

◆ 6. Matriz de confusión

La matriz de confusión permite analizar:

- verdaderos positivos
- falsos positivos
- falsos negativos
- verdaderos negativos

Esto es fundamental para interpretar el modelo desde el negocio, por ejemplo:

- clientes que el modelo cree que abandonarán pero no lo hacen
- clientes que abandonan sin ser detectados

La visualización facilita la interpretación más allá de los números.

◆ 7. Validación cruzada K-Fold

La validación cruzada divide el dataset en **k particiones** y:

- entrena el modelo k veces
- usa una partición distinta como validación en cada iteración
- promedia los resultados

En este práctico se utiliza:

- **StratifiedKFold**, que conserva la proporción de clases

Esto permite:

- obtener métricas más estables
- reducir dependencia de una sola partición
- estimar mejor la capacidad de generalización del modelo

◆ 8. GridSearchCV

Los modelos de Machine Learning poseen **hiperparámetros**, es decir, parámetros que:

- no se aprenden automáticamente
- deben definirse antes del entrenamiento

Ejemplos:

- profundidad de árboles
- cantidad de estimadores

- regularización

GridSearchCV permite:

- probar combinaciones específicas
- evaluar cada combinación con validación cruzada
- seleccionar automáticamente la mejor configuración

Es un método **exhaustivo**, útil cuando la grilla es pequeña.

◆ 9. RandomizedSearchCV

Cuando el espacio de hiperparámetros es grande, GridSearch puede resultar costoso.

RandomizedSearchCV:

- selecciona combinaciones al azar
- permite explorar espacios amplios
- reduce tiempo de cómputo
- suele obtener resultados similares o mejores

Por eso se utiliza frecuentemente en modelos como regresión logística o modelos complejos.

◆ 10. Comparación final de modelos

Una vez entrenados y ajustados los modelos:

- se evalúan **todos sobre el mismo conjunto test**
- se comparan métricas
- se selecciona el modelo final

Es importante entender que:

el conjunto test se utiliza una sola vez, al final.

Nunca debe participar en:

- entrenamiento
- validación
- selección de hiperparámetros

Reflexión final

Esta práctica busca que comprendas que:

- entrenar un modelo es solo una parte del proceso
- evaluar correctamente es igual o más importante
- un modelo sin validación adecuada puede parecer bueno... pero fallar en producción

Dominar validación, pipelines e hiperparámetros es lo que diferencia:

☞ **modelos académicos**

de

☞ **modelos profesionales de Data Science**

alizarse en un entorno Python con scikit-learn instalado. Se asume conocimiento básico de pandas y scikit-learn.

2 - Preparar el entorno: Python y Jupyter

Guía paso a paso para instalar paquetes recomendados, iniciar Jupyter y buenas prácticas para notebooks reproducibles, orientada a configurar un entorno de trabajo eficiente para proyectos de IA y ML.

¿Alguna vez te has preguntado cómo los científicos de datos y especialistas en IA organizan y ejecutan sus experimentos de manera eficiente y reproducible? La respuesta está en herramientas como Python y Jupyter Notebooks, que permiten combinar código, visualizaciones y documentación en un solo lugar. En esta unidad, aprenderás a preparar tu entorno de trabajo instalando Python y configurando Jupyter, la plataforma interactiva que revolucionó la forma de trabajar con datos y modelos de Machine Learning. Al finalizar, serás capaz de iniciar un notebook básico y entender las buenas prácticas para mantener tu

trabajo ordenado y reproducible, habilidades esenciales para cualquier profesional en Inteligencia Artificial y Machine Learning.



Introducción a Python y Jupyter Notebooks

Python es un lenguaje de programación ampliamente utilizado en la industria de IA y ML por su simplicidad y gran ecosistema de bibliotecas especializadas. Su sintaxis clara facilita la escritura y comprensión de código, ideal para quienes comienzan en programación.

Jupyter Notebooks es un entorno interactivo que permite crear y compartir documentos que contienen código ejecutable, ecuaciones, visualizaciones y texto narrativo. Esto facilita la experimentación, documentación y presentación de proyectos de datos.

Definición clave: *Un notebook es un archivo que combina código y texto en celdas que se pueden ejecutar de forma independiente, permitiendo un flujo de trabajo dinámico.*

Instalación y configuración básica

Para trabajar con Python y Jupyter, recomendamos usar **Anaconda**, una distribución que incluye Python y las bibliotecas más usadas en ciencia de datos, además de Jupyter.

Pasos básicos:

1. Descargar Anaconda desde <https://www.anaconda.com/products/distribution>.

2. Seguir el instalador según tu sistema operativo.
3. Abrir Anaconda Navigator y lanzar Jupyter Notebook.

Alternativamente, puedes instalar Python desde <https://www.python.org> y luego instalar Jupyter con el comando:

```
bash  
pip install notebook
```

Navegando en Jupyter Notebooks

- La interfaz web permite crear, abrir y organizar notebooks.
- Cada notebook está compuesto por celdas de código y texto (Markdown).
- Puedes ejecutar código celda por celda, facilitando pruebas y análisis iterativos.
- Los resultados de la ejecución se muestran inmediatamente debajo de la celda.
- Puedes agregar visualizaciones mediante bibliotecas como Matplotlib o Seaborn directamente en las celdas.
- Es posible exportar notebooks a formatos como HTML, PDF o scripts Python para compartir o documentar el trabajo.

Introducción a Python y Visual Studio Code

Python es un lenguaje de programación ampliamente utilizado en la industria de IA y ML por su simplicidad y gran ecosistema de bibliotecas especializadas. Su sintaxis clara facilita la escritura y comprensión de código, ideal para quienes comienzan en programación.

Visual Studio Code (VS Code) es un editor de código fuente ligero pero poderoso que, con las extensiones adecuadas, permite escribir, ejecutar y depurar código Python. Proporciona una experiencia integrada para desarrollo, experimentación y manejo de proyectos de datos.

Definición clave: Un archivo `.py` es un archivo de código Python, que se puede ejecutar directamente en VS Code o en la terminal, facilitando flujos de trabajo dinámicos y estructurados.

Instalación y configuración básica

Para trabajar con Python en VS Code recomendamos:

1. Descargar e instalar Python desde <https://www.python.org>.
2. Descargar e instalar Visual Studio Code desde <https://code.visualstudio.com>.
3. Abrir VS Code e instalar la extensión oficial de Python (Microsoft) desde la tienda de extensiones.
4. Opcionalmente, instalar Jupyter extension para trabajar con notebooks dentro de VS Code.

Pasos básicos para instalar Jupyter (si deseas usar notebooks):

```
bash  
pip install notebook
```

Navegando en Visual Studio Code

- VS Code permite crear, abrir y organizar archivos `.py` y notebooks `.ipynb`.
- Puedes escribir código Python en archivos `.py` o en celdas interactivas dentro de notebooks.
- La ejecución es posible celda por celda (para notebooks) o línea/archivo completo (para `.py`), facilitando pruebas y análisis iterativos.

Buenas prácticas para notebooks reproducibles

- **Documenta tu código:** usa celdas de texto para explicar el propósito y resultados.
- **Organiza el flujo:** estructura el notebook en secciones claras (importación, procesamiento, análisis).
- **Evita dependencias ocultas:** ejecuta todas las celdas en orden para asegurar reproducibilidad.
- **Guarda versiones:** usa control de versiones o guarda copias para seguimiento.

Estas prácticas son fundamentales para proyectos colaborativos y para mantener la integridad de tus experimentos en IA y ML.

En la industria de Inteligencia Artificial y Machine Learning, la capacidad de configurar un entorno de trabajo eficiente y reproducible es crucial. Equipos de datos en empresas tecnológicas, financieras o de salud utilizan Python y Jupyter para desarrollar modelos, analizar datos y compartir resultados con claridad. Por ejemplo, un equipo de analistas puede colaborar en un notebook para explorar datos de clientes y entrenar modelos predictivos, asegurando que cada paso esté documentado y pueda ser replicado o auditado.

Preparar tu entorno correctamente desde el inicio te permitirá enfocarte en el desarrollo de modelos y análisis sin perder tiempo en problemas técnicos. Además, dominar Jupyter Notebooks te facilitará la transición a módulos posteriores donde entrenarás y evaluarás modelos de Machine Learning con bibliotecas como scikit-learn y pandas.

Instrucciones:

Práctica: Configura tu entorno Python y crea tu primer Jupyter Notebook en Visual Studio Code

1. **Instala Python:** Descarga e instala Python desde la página oficial python.org si no lo tienes instalado.
2. **Instala Visual Studio Code:** Descarga e instala Visual Studio Code desde code.visualstudio.com.
3. **Instala la extensión de Python:** Abre VS Code, ve a la pestaña de extensiones (icono de cubo) y busca “Python”. Instala la extensión oficial de Microsoft.
4. **Instala la extensión de Jupyter:** De modo similar, busca e instala la extensión “Jupyter”.
5. **Crea un nuevo archivo notebook:** En VS Code, presiona Ctrl+Shift+P, escribe “Jupyter: Create New Jupyter Notebook” y selecciónalo para crear un nuevo archivo `.ipynb`.
6. **Escribe y ejecuta código básico:** En la primera celda, escribe `print("¡Hola, Jupyter!")` y presiona Shift+Enter para ejecutar la celda.
7. **Agrega una celda de texto:** Agrega una nueva celda, cambia el tipo a Markdown desde el menú de la celda y escribe una breve descripción de lo que hiciste.
8. **Guarda tu notebook:** Usa Archivo > Guardar como para guardar el archivo con un nombre descriptivo en tu carpeta de trabajo.

Recuerda seguir las buenas prácticas aprendidas: documenta cada paso y organiza tu notebook para facilitar su lectura y reproducción.

Al completar esta práctica, habrás configurado tu entorno en Visual Studio Code y creado un notebook básico listo para tus proyectos de IA y ML.

Entregable:

Un notebook de Jupyter (`.ipynb`) que contenga:

- Una celda de código con la ejecución exitosa de `print("¡Hola, Jupyter!")`.
- Una celda de texto en Markdown explicando brevemente el propósito del notebook.
- El archivo debe estar guardado con un nombre claro, por ejemplo, `primer_notebook.ipynb`.

Este entregable demostrará que has configurado correctamente tu entorno y sabes crear y ejecutar un notebook básico siguiendo buenas prácticas.

3 - Feature engineering: creación y transformación de variables

Ejercicio práctico para crear nuevas variables (features) a partir de datos existentes, codificar variables categóricas mediante one-hot y label encoding, y escalar variables numéricas. Este ejercicio refuerza la comprensión y aplicación básica del feature engineering en Python, usando solo estructuras y funciones estándar para simular transformaciones comunes en ML.

Diagnóstico de calidad de datos y limpieza

En este ejercicio, aplicarás técnicas básicas de limpieza de datos usando Python y pandas para preparar un dataset para análisis y modelado. Trabajaremos con un dataset simulado que contiene valores faltantes, outliers, tipos incorrectos y texto inconsistente.

Investigá y aplicá las técnicas que conoces para terminar el código que aparece al final en la sección de la actividad.

Sugerencia: Podes copiar y pegar el código como está en tu entorno preferido o usar Google Colab, sacando las líneas de código finales

```
import sys
# Módulo para leer stdin (útil si el CSV se pasa por tubería) # noqa: E402
import io
# Utilizado para manejar cadenas como archivos (fallback)
# noqa: E402
import pandas as pd
# Biblioteca principal para manipulación de datos
# noqa: E402
import numpy as np
# Utilidades numéricas (medianas, máscaras, etc.)
# noqa: E402
import re
# Expresiones regulares para normalizar texto
# noqa: E402
def create_sample_csv():
    # Función que genera un CSV de ejemplo cuando no
    hay stdin # noqa: E402    sample =
    """id,edad,ingresos,fecha_registro,nombre,comentario1,25,50000,2021-01-05,
Juan ,Buen cliente2,32,,2021-03-10,María, Compra
grande3,47,120000,not_a_date,ANA,DEVOLVIÓ producto4,,110000,2021-06-01,
pedro ,pedido urgente5,62,150000,2021/07/15,Ana ,Interesado6,23,48000,2021-
08-20,maría, revisión pendiente7,43,90000,2021-09-30,JUAN,
    buen cliente8,36,72000,2021-10-12,sofia, buen historial9,29,58000,2021-
11-05,Sofía, compra repetida10,41,85000,2021-12-01, ,sin nombre"""
    return io.StringIO(sample)
# Devuelve un objeto file-like con el CSV de ejemplo
```

```

def detect_numeric_columns(df):
    # Detecta columnas numéricas de forma robusta
    # noqa: E402    num_cols =
df.select_dtypes(include=[np.number]).columns.tolist() # Columnas ya
numéricas # noqa: E402
    # Intentamos convertir columnas que parecen numéricas pero son object
    (ej. '1', '2,000')
    for col in df.columns:
        if col not in num_cols:
            # Heurística: si más del 70% de valores convertibles a float ->
            tratar como numérica
            sample = df[col].dropna().astype(str).head(200)
            # Muestra de valores para prueba # noqa: E402            convertible =
            sample.apply(lambda x: bool(re.match(r'^[\s\-\+]*\d+([.]\d+)?\s*$', x))) #
            noqa: E402            if len(sample) > 0 and convertible.sum() /
            len(sample) >= 0.7:
                num_cols.append(col)
    # Añadimos a numéricas si pasa la heurística # noqa: E402    return
    num_cols

def coerce_numeric(df, cols):
    # Convierte columnas detectadas a float (handling
    de comas) # noqa: E402    for c in cols:
        df[c] = df[c].astype(str).str.replace(',', '.')
    # Reemplazar coma decimal por punto si existe # noqa: E402            df[c] =
    pd.to_numeric(df[c], errors='coerce')
    # Forzar conversión; errores -> NaN
    # noqa: E402    return df

def impute_median(df, cols):
    # Imputa valores faltantes por la mediana (robusto
    frente a outliers) # noqa: E402    for c in cols:
        med = df[c].median(skipna=True)
    # Calcula mediana ignorando NaN
    # noqa: E402            df[c] = df[c].fillna(med)
    # Rellena NaN con la mediana
    # noqa: E402    return df

def cap_outliers_iqr(df, cols):
    # Detecta outliers por IQR y los "capea" al límite más
    cercano # noqa: E402    for c in cols:
        q1 = df[c].quantile(0.25)
    # Primer cuartil
    # noqa: E402            q3 = df[c].quantile(0.75)
    # Tercer cuartil
    # noqa: E402            iqr = q3 - q1
    # Rango intercuartílico
    # noqa: E402            if pd.isna(iqr) or iqr == 0:
                continue
    # Saltar columnas sin variabilidad
    # noqa: E402            lower = q1 - 1.5 * iqr
    # Límite inferior
    # noqa: E402            upper = q3 + 1.5 * iqr
    # Límite superior
    # noqa: E402
        # Reemplazamos valores por los límites cuando están fuera del rango
        df[c] = np.where(df[c] < lower, lower, df[c]) # Valores debajo del límite -
        > lower
        # noqa: E402            df[c] = np.where(df[c] > upper, upper, df[c]) # Valores
        arriba del límite -> upper
    # noqa: E402    return df

def coerce_datetime(df, candidates):
    # Intenta convertir columnas candidatas a datetime
    # noqa: E402    for c in candidates:

```

```

        # Intento flexible con pandas; errores->NaT          df[c] =
pd.to_datetime(df[c], errors='coerce', infer_datetime_format=True)
    return df
def normalize_text_columns(df):
    # Normaliza texto: minúsculas, strip y colapsa
    espacios # noqa: E402    for c in df.select_dtypes(include=['object',
'string']).columns:
        # Evitar tocar columnas que eran IDs numéricos convertidos a string
        (heurística simple)
        if c.lower() in ['id', 'identifier', 'index']:
            continue
        s = df[c].astype(str)
# Forzar a string para aplicar métodos
# noqa: E402          s = s.str.strip()
# Quitar espacios al inicio/fin
# noqa: E402          s = s.str.replace(r'\s+', ' ', regex=True)
        # Colapsar espacios múltiples en uno
# noqa: E402          s = s.replace({'nan': None})
# Evitar que 'nan' string quede como texto
# noqa: E402
        # Normalización de mayúsculas/minúsculas respetando acentos
        s = s.str.normalize('NFKC')
# Normaliza unicode (compatibilidad)
# noqa: E402          s = s.str.lower()
# Pasa a minúsculas
# noqa: E402
        # Aplicar reemplazos típicos (opcional): e.g. nombres vacíos -> NaN
s = s.replace(r'^\s*$', np.nan, regex=True)
        # Strings vacíos -> NaN
# noqa: E402          df[c] = s
# Asignar de vuelta al DataFrame
# noqa: E402    return df
def solve():
    # Función principal que orquesta la
    limpieza y la salida # noqa: E402
    # Lectura del CSV desde stdin (útil para pipelines). Si no hay stdin,
    usamos el CSV de ejemplo.    raw = sys.stdin.read()
    # Leer toda la entrada estándar
    # noqa: E402    if raw.strip() == "":
        # Si stdin está vacío -> fallback a dataset
        simulado
        # noqa: E402          csv_file = create_sample_csv()
    # Generar CSV de ejemplo como file-like
    # noqa: E402
    else:
        csv_file = io.StringIO(raw)
    # Convertir el contenido de stdin en file-like
    # noqa: E402
        df = pd.read_csv(csv_file)
    # Cargar CSV en DataFrame
    # noqa: E402
    # -----
    # Diagnóstico inicial
    # -----
    print("# Diagnóstico inicial")
    # Título del bloque diagnóstico
    # noqa: E402
    print("Dimensiones (filas, columnas):", df.shape) # Mostrar tamaño del
    dataset
    # noqa: E402
    print("\nValores nulos por columna:")

```

```

# Encabezado para nulos
# noqa: E402
print(df.isnull().sum())
# Cuenta de valores nulos por columna
# noqa: E402
print("\nTipos detectados por pandas:")
# Mostrar tipos actuales
# noqa: E402
print(df.dtypes)
# Tipos de datos por columna
# noqa: E402
print("\nPrimeras filas (raw):")
# Mostrar primeras filas antes de transformaciones
# noqa: E402
print(df.head().to_string(index=False))
# Imprimir primeras filas en formato legible
# noqa: E402
# -----
# 1) Detectar y coaccionar columnas numéricas
# -----
numeric_candidates = detect_numeric_columns(df) # Encuentra columnas
numéricas o convertibles
# noqa: E402
print("\nColumnas numéricas detectadas o convertibles:",
numeric_candidates) # noqa: E402df = coerce_numeric(df,
numeric_candidates) # Forzar conversión a numérico (NaN si falla)
# noqa: E402
# -----
# 2) Imputar valores faltantes en numéricas con mediana
# -----
df = impute_median(df, numeric_candidates) # Rellenar NaN en numéricas con
la mediana
# noqa: E402
print("\nDespués de imputación (valores nulos por columna):") # noqa: E402
print(df[numeric_candidates].isnull().sum()) # Verificar que no queden NaN
en numéricas
# noqa: E402
# -----
# 3) Detectar y capear outliers por IQR en numéricas
# -----
df = cap_outliers_iqr(df, numeric_candidates) # Reemplazar outliers por
límites IQR
# noqa: E402
print("\nEstadísticas numéricas (post-capping):") # noqa: E402
print(df[numeric_candidates].describe().T) # Mostrar estadísticas
resumidas por columna
# noqa: E402
# -----
# 4) Corregir tipos de datos incorrectos (fechas, categorías)
# -----
# Intentamos convertir columnas que contienen la palabra 'fecha' o 'date' a
datetime
date_cols = [c for c in df.columns if 'fecha' in c.lower() or
'date' in c.lower()] # noqa: E402if date_cols:
    df = coerce_datetime(df, date_cols) # Forzar conversión a datetime
(NaT si falla)
# noqa: E402    print("\nColumnas convertidas a datetime:", date_cols) #
noqa: E402    print(df[date_cols].head()) # Ver primeras filas de las
columnas de fecha
# noqa: E402
# -----
# 5) Normalizar texto en columnas object/string

```



```

# -----
df = normalize_text_columns(df) # Aplicar lower/strip/colapsar-espacios
# noqa: E402
print("\nMuestra de columnas texto normalizadas:") # noqa: E402
text_cols = df.select_dtypes(include=['object', 'string']).columns.tolist() # noqa: E402
print(df[text_cols].head().to_string(index=False)) # Mostrar ejemplo de texto normalizado
# noqa: E402
# -----
# 6) Correcciones puntuales y heurísticas finales
# -----
# Ejemplos:
# - Si hay columna 'nombre' y valores repetidos con diferenciales de acento, podemos armonizarif 'nombre' in df.columns:
    df['nombre'] = df['nombre'].str.normalize('NFKD').str.encode('ascii', errors='ignore').str.decode('ascii') # noqa: E402
df['nombre'] = df['nombre'].str.replace(r'\s+', ' ', regex=True).str.strip().replace(' ', np.nan) # noqa: E402
# - Si existe columna 'id' y fue leída como float por pandas (ej. 1.0), la convertimos a enteroif 'id' in df.columns and
pd.api.types.is_float_dtype(df['id']):
    df['id'] = df['id'].fillna(0).astype(int) # Convertir IDs a int (completar NaN con 0 antes)
    # noqa: E402
# -----
# 7) Resultado final: imprimir CSV limpio sin índice
# -----
print("\nDataset limpio (CSV):") # Indicador antes del dump CSV
# noqa: E402
print(df.to_csv(index=False)) # Imprime el CSV limpio en stdout (útil para redirecciones) # noqa: E402

# -----
# IMPORTS
# -----
import numpy as np
# Manejo de arrays y operaciones numéricas
import pandas as pd
# DataFrame para presentación ordenada de resultados
import matplotlib.pyplot as plt
# Gráficos estáticos (ROC, matriz de confusión)
from sklearn.datasets import make_classification # Crear dataset sintético controlado
from sklearn.model_selection import train_test_split, StratifiedKFold # Split y CV estratificada
from sklearn.linear_model import LogisticRegression # Modelo simple, interpretable y con pruebas
from sklearn.ensemble import RandomForestClassifier # Modelo más flexible para comparar
from sklearn.metrics import ( # Métricas y utilidades
    accuracy_score, precision_score, recall_score, f1_score,
    confusion_matrix, classification_report,
    roc_curve, roc_auc_score)
# -----
# 1) Generar dataset sintético (con desbalance para ilustrar limitaciones de Accuracy)
# -----
# make_classification permite controlar n_samples, n_features, n_classes, pesos de clase, ruido, etc.X, y = make_classification(

```

```

    n_samples=1000,          # número de observaciones
    n_features=10,           # número de variables predictoras
    n_informative=3,         # cuántas features realmente informativas
    n_redundant=2,           # features redundantes (combinaciones
lineales)
    n_repeated=0,            # features repetidas
    n_classes=2,             # problema binario
    weights=[0.8, 0.2],      # proporción de clases: clase 0 = 80%, clase 1
= 20% (desequilibrado)
    class_sep=1.0,           # separabilidad entre clases (mayor => más
fácil)
    random_state=42
# semilla para reproducibilidad)
# Mostrar balance de clases (útil para discusión en clase)
unique, counts = np.unique(y, return_counts=True)
    # contar ejemplos por clase
print("Distribución de clases (label:count):", dict(zip(unique, counts)))
# -----
# 2) Dividir en train / test de forma estratificada (mantener proporción de
clases)
# -----
X_train, X_test, y_train, y_test = train_test_split(
    X, y,
    test_size=0.25,          # 25% para prueba
    stratify=y,              # mantiene la proporción de clases en ambos
sets
    random_state=42)
# -----
# 3) Entrenar dos modelos (uno simple, otro más complejo) para comparar
métricas
# -----
# Modelo 1: Regresión logística (buena línea base; devuelve probabilidades)
model_lr = LogisticRegression(max_iter=1000, class_weight='balanced',
random_state=42)
# class_weight='balanced' ajusta la penalización por clase según su
frecuencia (útil en desbalance)
# Modelo 2: Random Forest (no lineal, robusto, menos necesidad de
ingeniería)
model_rf = RandomForestClassifier(n_estimators=100, random_state=42,
class_weight='balanced')
# Ajustar modelos
model_lr.fit(X_train, y_train)
    # entrenar regresión logística sobre los datos de
entrenamiento
model_rf.fit(X_train, y_train)
    # entrenar random forest sobre los datos de entrenamiento
# -----
# 4) Predicciones y probabilidades (necesarias para ROC)
# -----
# Predicciones de clase
y_pred_lr = model_lr.predict(X_test)    #
predicciones discretas (0/1) con LR
y_pred_rf = model_rf.predict(X_test)    #
predicciones discretas (0/1) con RF
# Probabilidades para la clase positiva (útiles para curva ROC y AUC)
y_proba_lr = model_lr.predict_proba(X_test)[:, 1] # probabilidad de clase
1 para LR
y_proba_rf = model_rf.predict_proba(X_test)[:, 1] # probabilidad
de clase 1 para RF
# -----
# 5) Cálculo de métricas clásicas (Accuracy, Precision, Recall, F1)
# -----
# Función auxiliar para calcular y devolver métricas en un diccionario
def compute_metrics(y_true, y_pred):    """Devuelve accuracy, precision,
recall, f1 y confusion matrix."""
    acc = accuracy_score(y_true, y_pred)
# proporción de aciertos

```

```

    prec = precision_score(y_true, y_pred, zero_division=0)
    # precision (evita error si no hay positivos)
    rec = recall_score(y_true, y_pred, zero_division=0)
# recall (sensibilidad)
    f1 = f1_score(y_true, y_pred, zero_division=0)
# F1 (media armónica)
    cm = confusion_matrix(y_true, y_pred)
# matriz de confusión 2x2    return {'accuracy': acc, 'precision': prec,
'recall': rec, 'f1': f1, 'confusion_matrix': cm}
# Calcular métricas para ambos modelos
metrics_lr = compute_metrics(y_test,
y_pred_lr)
metrics_rf = compute_metrics(y_test, y_pred_rf)
# Mostrar métricas en formato legible (DataFrame para la clase)
df_metrics = pd.DataFrame([metrics_lr, metrics_rf],
index=['LogisticRegression', 'RandomForest'])
# Sacar la matriz de confusión de la tabla para evitar formato no legible;
la imprimimos aparte
df_metrics_no_cm =
df_metrics.drop(columns=['confusion_matrix'])
print("\nMétricas resumen (sin matriz de confusión):")
print(df_metrics_no_cm.round(4))
# Imprimir matrices de confusión por separado con explicación
print("\nMatriz de confusión - Logistic Regression (filas=verdadero,
columnas=predicho):")
print(metrics_lr['confusion_matrix'])
print("\nMatriz de confusión - Random Forest (filas=verdadero,
columnas=predicho):")
print(metrics_rf['confusion_matrix'])
# -----
# 6) Reporte detallado (precision/recall/f1 por clase) usando
classification_report
# -----
print("\nClassification report - Logistic Regression:")
print(classification_report(y_test, y_pred_lr, digits=4))
print("\nClassification report - Random Forest:")
print(classification_report(y_test, y_pred_rf, digits=4))
# -----
# 7) Curva ROC y AUC (capacidad discriminativa global)
# -----
# Calcular curva ROC (tpr, fpr) y AUC para ambos modelos
fpr_lr, tpr_lr, _ = roc_curve(y_test, y_proba_lr)    # puntos ROC para LR
auc_lr = roc_auc_score(y_test, y_proba_lr)
# AUC para LR
fpr_rf, tpr_rf, _ = roc_curve(y_test, y_proba_rf)    # puntos ROC para
RF
auc_rf = roc_auc_score(y_test, y_proba_rf)
# AUC para RF
# Graficar ROC en la misma figura para comparar (buena discusión en clase)
plt.figure(figsize=(8,6))
# tamaño de la figura
plt.plot(fpr_lr, tpr_lr, label=f'LogReg (AUC = {auc_lr:.3f})')    # curva LR
con AUC en la leyenda
plt.plot(fpr_rf, tpr_rf, label=f'RandomForest (AUC = {auc_rf:.3f})')    #
curva RF con AUC
plt.plot([0,1], [0,1], 'k--', label='Random guess')    # línea diagonal =
clasificador aleatorio
plt.xlabel('False Positive Rate (FPR)')
# etiqueta eje X
plt.ylabel('True Positive Rate (TPR)')
# etiqueta eje Y
plt.title('ROC Curve - comparación de modelos')
# título del gráfico
plt.legend(loc='lower right')

```

```

# leyenda en la esquina inferior derecha
plt.grid(True)
# grilla para mejor lectura
plt.show()
# mostrar la figura
# -----
# 8) Discusión e interpretación imprimible (texto para clase)
# -----
print("\n--- Interpretación (guía para docentes/estudiantes) ---")
print("1) Accuracy: mide aciertos totales. Útil pero engañosa en datasets desbalanceados.")
print("2) Precision: de todas las predicciones positivas, cuántas son correctas (útil si FP cuesta).")
print("3) Recall: de todos los positivos reales, cuántos detectamos (útil si FN cuesta).")
print("4) F1: balance entre precision y recall; recomendable cuando ambas importan.")
print("5) AUC: capacidad global del modelo para ordenar ejemplos por probabilidad de pertenecer a la clase positiva.")
print("\nEjemplo específico (nuestra simulación):")
print("- Observá las métricas por modelo y la matriz de confusión para entender errores tipo FP/FN.")
print("- Compara AUC para ver qué modelo discrimina mejor en todos los umbrales.")
print("- En casos desbalanceados, preferimos revisar precision/recall/F1 y la matriz de confusión antes que solo accuracy.")
# -----
# 9) (Opcional) Validación cruzada estratificada para estimar estabilidad de F1
# -----
cv = StratifiedKFold(n_splits=5, shuffle=True, random_state=42) # CV estratificado
# Evaluar F1 mediante CV en el conjunto completo (buena práctica para pequeñas muestras)
from sklearn.model_selection import cross_val_score
cv_scores_lr = cross_val_score(model_lr, X, y, cv=cv, scoring='f1')
# f1 para LR
cv_scores_rf = cross_val_score(model_rf, X, y, cv=cv, scoring='f1')
# f1 para RF
print(f"\nCross-val F1 (5 folds) - LogReg: mean={cv_scores_lr.mean():.4f}, std={cv_scores_lr.std():.4f}")
print(f"Cross-val F1 (5 folds) - RF: mean={cv_scores_rf.mean():.4f}, std={cv_scores_rf.std():.4f}")

python
if __name__ == '__main__':
    solve()

```

Objetivos del ejercicio

- Detectar y tratar valores faltantes con imputación simple.
- Identificar outliers usando el método del rango intercuartílico (IQR).
- Corregir tipos de datos incorrectos para asegurar análisis correcto.
- Normalizar texto para uniformidad.

Instrucciones

Se te proporcionará un dataset en formato CSV (simulado como entrada estándar). Debes:

1. Cargar el dataset usando pandas.
 2. Detectar y reemplazar valores faltantes en columnas numéricas con la mediana.
 3. Detectar outliers en columnas numéricas usando IQR y reemplazarlos con el valor del límite más cercano.
 4. Convertir columnas con tipos incorrectos a su tipo correcto.
 5. Normalizar texto en columnas de tipo string (minúsculas, sin espacios extra).
 6. Imprimir el dataset limpio en formato CSV sin índice.
-

Recomendaciones

- Usa pandas para manipulación de datos.
- Para imputación, la mediana es robusta frente a outliers.
- Para outliers, calcula Q1, Q3 y define límites inferior y superior.
- Para texto, usa métodos de string de pandas.

4 - Tipos de aprendizaje: supervisado, no supervisado y por refuerzo

Explora los tres paradigmas principales del aprendizaje automático: supervisado, no supervisado y por refuerzo. Aprende sus características, ejemplos prácticos y limitaciones para saber cuándo aplicar cada uno en proyectos reales de IA.

¿Alguna vez te has preguntado cómo una aplicación puede reconocer tu voz, recomendarte una película o incluso jugar videojuegos mejor que un humano? Detrás de estas capacidades está el aprendizaje automático (ML), que permite a las máquinas aprender de los datos y mejorar con la experiencia. En esta unidad, descubrirás los tres tipos principales de aprendizaje automático: supervisado, no supervisado y por refuerzo. Entenderás qué los diferencia, cómo funcionan y en qué situaciones se aplican, preparando el terreno para que puedas elegir el enfoque adecuado en tus futuros proyectos de IA.

Tipos de aprendizaje en Machine Learning

El aprendizaje automático se basa en diferentes paradigmas que definen cómo los modelos aprenden de los datos. A continuación, presentamos los tres principales:



El aprendizaje automático se basa en diferentes paradigmas que definen cómo los modelos aprenden de los datos. A continuación, presentamos los tres principales:

1. Aprendizaje Supervisado

El **aprendizaje supervisado** es un tipo de ML donde el modelo aprende a partir de datos etiquetados, es decir, cada entrada tiene una salida conocida. El objetivo es que el modelo aprenda la relación entre entradas y salidas para predecir resultados en datos nuevos.

- **Ejemplo:** Clasificar correos electrónicos como "spam" o "no spam" usando un conjunto de correos previamente etiquetados.
- **Aplicaciones comunes:** Reconocimiento de voz, detección de fraudes, diagnóstico médico.

Definición técnica: Un conjunto de datos ($D = \{(x_i, y_i)\}$) donde (x_i) es la entrada y (y_i) la etiqueta o salida.

Ejemplo de aprendizaje supervisado en código:

```
# Importaciones: scikit-learn para procesamiento, modelo y evaluación
from sklearn.model_selection import train_test_split, cross_val_score,
StratifiedKFold
from sklearn.feature_extraction.text import TfidfVectorizer # TF-IDF en
lugar de conteos
from sklearn.linear_model import LogisticRegression
# clasificador con class_weight
from sklearn.pipeline import make_pipeline
from sklearn.metrics import accuracy_score, classification_report,
confusion_matrix
import numpy as np
import warnings.filterwarnings("ignore") # evitar warnings que
distraen en ejemplos pequeños
# Datos de ejemplo (pequeño toy dataset)
X = [
    "Gana dinero rápido ahora",
    "Reunión mañana a las 10am",
    "Oferta exclusiva: compra 1 lleva 2",
    "¿Puedes revisar el informe adjunto?",
    "Última llamada: premio para ti",
    "Proyecto: actualización del cronograma",
    "Haz clic aquí para reclamar tu recompensa",
    "Confirmación de tu cita médica",
    "Descuento especial solo hoy",
    "Factura y detalles de pago"]
y = ["spam", "ham", "spam", "ham", "spam", "ham", "spam", "ham", "spam",
"ham"]
# Separar en entrenamiento y prueba
# stratify=y mantiene la proporción de clases en train/test (útil cuando
hay desbalance)
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.3, random_state=42, stratify=y)
# Pipeline: TF-IDF vectorizer + Logistic Regression con
class_weight='balanced'# - TfidfVectorizer: convierte texto a vectores
ponderando por frecuencia inversa de documento (da menos peso a palabras
muy comunes)
# - ngram_range=(1,2): usa uni- y bi-gramas para capturar pares de palabras
(pocos parámetros pero más contexto)
# - min_df=1 (por defecto aquí, dataset pequeño). En datasets reales
conviene usar min df más alto.pipeline = make_pipeline(
    TfidfVectorizer(ngram_range=(1,2)), #
transforma texto a features TF-IDF LogisticRegression(max_iter=1000,
class_weight='balanced') # modelo lineal con contrabalanceo de clases)
# Entrenar el pipeline en el set de entrenamientopipeline.fit(X_train, y_train)
# Evaluación en set de prueba (inferir y medir)
y_pred = pipeline.predict(X_test)
acc = accuracy_score(y_test, y_pred)
report = classification_report(y_test, y_pred, digits=3)
cm = confusion_matrix(y_test, y_pred, labels=["spam", "ham"])
print("=== Evaluación en set de prueba ===")
print(f"Accuracy: {acc:.3f}")
print("\nClassification report:\n", report)
print("Matriz de confusión (filas=verdadero, columnas=predicho):\n", cm)
# Cross-validation estratificada (estimación más estable con pocos datos)
# Usamos f1_macro para balancear el efecto de ambas clasescv =
StratifiedKFold(n_splits=3, shuffle=True, random_state=42)
scores = cross_val_score(pipeline, X, y, cv=cv, scoring='f1_macro') #
f1_macro balancea por clase
```

```
print(f"\nCross-val f1_macro (3 folds): mean={scores.mean():.3f},
std={scores.std():.3f}")
# Probar el modelo con nuevos ejemplos prácticosnuevos = [
    "Felicitaciones, has ganado un premio",    # probable spam
    "Recordatorio: pago pendiente de tu factura" # probable ham (pero puede
confundirse)]
preds = pipeline.predict(nuevos)
probs = pipeline.predict_proba(nuevos) # probabilidades por clase (útil
para umbrales)
for texto, etiqueta, prob in zip(nuevos, preds, probs):
    print(f"\nTexto: '{texto}'\nPredicción: {etiqueta}\nProbabilidades
[spam, ham]: {prob.round(3)}")
```

=== Evaluación en set de prueba ===

Accuracy: 0.667

Classification report:

Matriz de confusión (filas=verdadero, columnas=predicho):

```
[[1 0]
 [1 1]]
```

Cross-val f1_macro (3 folds): mean=0.689, std=0.031

Texto: 'Felicitaciones, has ganado un premio'

Predicción: spam

Probabilidades [spam, ham]: [0.458 0.542]

Texto: 'Recordatorio: pago pendiente de tu factura'

Predicción: ham

Probabilidades [spam, ham]: [0.541 0.459]

```
Matriz de confusión (filas=verdadero, columnas=predicho):
[[1 0]
 [1 1]]
```

Cross-val f1_macro (3 folds): mean=0.689, std=0.031

Texto: 'Felicitaciones, has ganado un premio'

Predicción: spam

Probabilidades [spam, ham]: [0.458 0.542]

Texto: 'Recordatorio: pago pendiente de tu factura'

Predicción: ham

Probabilidades [spam, ham]: [0.541 0.459]

2. Aprendizaje No Supervisado

El **aprendizaje no supervisado** trabaja con datos sin etiquetas. El objetivo es encontrar patrones, estructuras o agrupaciones dentro de los datos.

- **Ejemplo:** Agrupar clientes según sus comportamientos de compra para segmentar mercados.
- **Aplicaciones comunes:** Análisis de clusters, reducción de dimensionalidad, detección de anomalías.

Definición técnica: Dado un conjunto de datos ($D = \{x_i\}$) sin etiquetas, el modelo busca estructuras internas.

notebook-python

```
# 1) Importaciones: librerías básicas para crear datos, escalar,
clusterizar, reducir dimensión y graficar
import numpy as np
import pandas as pd
from sklearn.datasets import make_blobs
# crear datos sintéticos (solo para demo)
from sklearn.preprocessing import StandardScaler # normalizar features
antes de clustering
from sklearn.decomposition import PCA
# reducir a 2D para visualización
from sklearn.cluster import KMeans
# algoritmo de clustering
from sklearn.metrics import silhouette_score
# métrica para validar clusters
import matplotlib.pyplot as plt%matplotlib inline
# Crear un dataset sintético (D = {x_i}) sin usar las etiquetas generadas
# make_blobs genera coordenadas con estructuras naturales (útil para
demostrar métodos)
X, _ = make_blobs(n samples=150, centers=3, cluster std=1.0,
random_state=42)
# Nota: el segundo valor "_" contiene etiquetas verdaderas creadas por
make_blobs, # pero en aprendizaje no supervisado no se usan para
entrenar el modelo.
#Convertir a DataFrame para facilitar inspección (opcional)
df = pd.DataFrame(X, columns=["feature_1", "feature_2"])
print("Primeras filas del dataset (sin etiquetas):")
print(df.head())
# Escalado: StandardScaler centra y escala las features a media 0 y
varianza 1.
# Es importante porque KMeans usa distancias euclidianas sensibles a la
escala.scaler = StandardScaler()
X_scaled = scaler.fit_transform(df.values)
# Selección del número de clusters (k): aquí elegimos k=3 (pocos
parámetros)
k = 3kmeans = KMeans(n_clusters=k, random_state=42, n_init=10) # n_init
controla re-inicios para estabilidad
# Entrenar (fit) el modelo sobre X_scaled: el algoritmo busca centros que
minimicen la varianza internakmeans.fit(X_scaled)
#Obtener etiquetas de cluster asignadas (predicciones no supervisadas)
labels = kmeans.labels_
# Evaluación interna: Silhouette score (entre -1 y 1) – cuanto más alto,
mejor separación de clusterssil_score = silhouette_score(X_scaled, labels)
print(f"\nSilhouette score (medida interna de calidad): {sil_score:.3f}")
# amaño de cada cluster (útil para entender balanceo)
unique, counts = np.unique(labels, return_counts=True)
print("\nTamaño de clusters (cluster_id: count):")
for u, c in zip(unique, counts):
    print(f" {u}: {c}")
```

```
# Reducir a 2D para graficar (si el dataset tuviera más dimensiones
usaríamos PCA igualmente)
pca = PCA(n_components=2, random_state=42)
X_pca = pca.fit_transform(X_scaled)
# Proyectar centros de KMeans al espacio PCA para visualizarlos
centers_pca = pca.transform(centers)
# Graficar resultados: puntos coloreados por su cluster y centros marcados
plt.figure(figsize=(8,6))
for cluster_id in range(k):
    idx = labels == cluster_id
    plt.scatter(X_pca[idx, 0], X_pca[idx, 1],
label=f"Cluster {cluster_id}", alpha=0.6)
plt.scatter(centers_pca[:,0], centers_pca[:,1], marker='X', s=200,
c='black', label='Centros')
plt.title(f'KMeans clustering (k={k}) proyectado con PCA - Silhouette:
{sil_score:.3f}')
plt.xlabel('PCA 1')
plt.ylabel('PCA 2')
plt.legend()
plt.grid(True)
plt.show()
# Ejemplo práctico: añadir la etiqueta de cluster al DataFrame (útil para
segmentación y análisis)
df['cluster'] = labels
print("\nEjemplo de segmentación (primeras filas con cluster asignado):")
print(df.head())
```

Primeras filas del dataset (sin etiquetas):

```
feature_1 feature_2
0 -2.545024 10.578930
1 -3.348415 8.705074
2 -6.598635 -7.502809
3 6.526065 2.147747
4 -6.364592 -6.366324
```

Silhouette score (medida interna de calidad): 0.846

Tamaño de clusters (cluster_id: count):

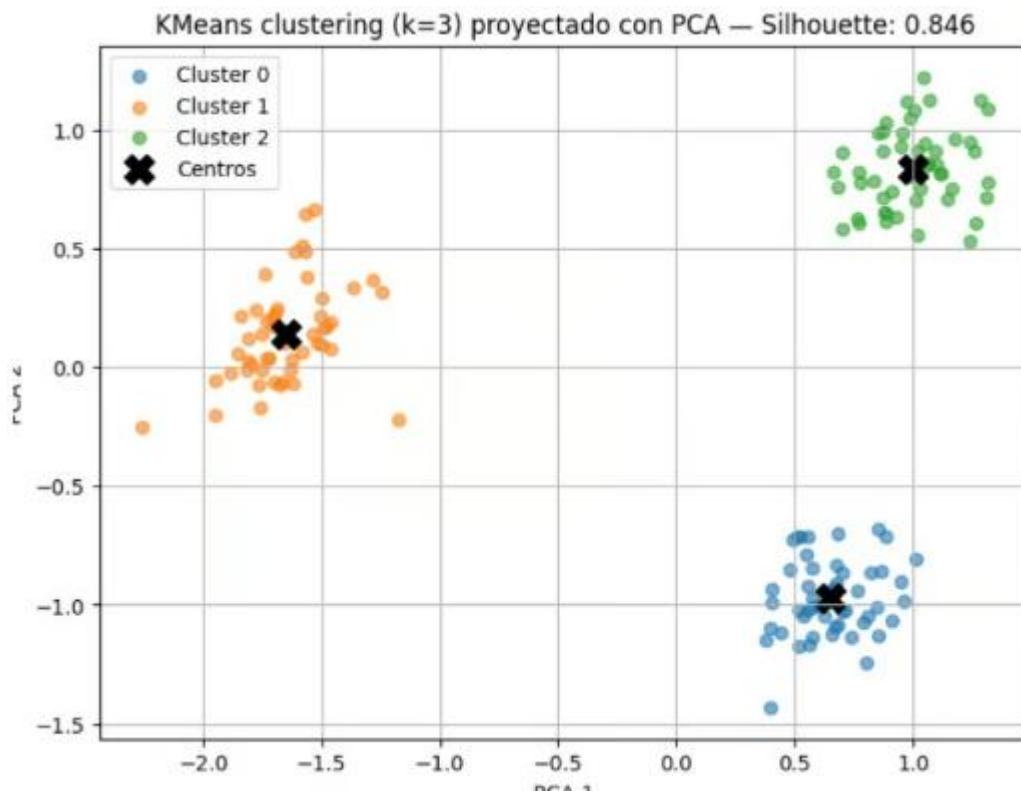
```
0: 50
1: 50
2: 50
```

Ejemplo de segmentación (primeras filas con cluster asignado):

```
feature_1 feature_2 cluster
0 -2.545024 10.578930 0
1 -3.348415 8.705074 0
2 -6.598635 -7.502809 1
```

3 6.526065 2.147747 2

4 -6.364592 -6.366324 1



3. Aprendizaje por Refuerzo

El **aprendizaje por refuerzo** implica que un agente aprende a tomar decisiones mediante prueba y error, recibiendo recompensas o penalizaciones según sus acciones.

- **Ejemplo:** Un robot que aprende a caminar optimizando sus movimientos para evitar caídas.
- **Aplicaciones comunes:** Juegos, robótica, sistemas de recomendación adaptativos.

Definición técnica: El agente interactúa con un entorno (E), tomando acciones (a_t) en estados (s_t), y recibe recompensas (r_t).

notebook-python

```
# El agente decide si recomendar o no, según el "estado" del usuario.
```

```
import numpy as np
import random
```

```
class RecommendationEnv:
    """ Estados del usuario: 0 = Desinteresado
    1 = Neutral 2 = Interesado
    Acciones: 0 = No recomendar 1 = Recomendar
    Recompensas: +1 si recomienda y el usuario está interesado -1 si
    recomienda a un usuario desinteresado (malas recomendaciones) 0 en otros
    casos """
    def __init__(self):
        self.reset()
```

```

def reset(self):          # Estado inicial del usuario, elegido aleatoriamente
(simulación realista)    self.state = np.random.choice([0, 1, 2], p=[0.4, 0.4,
0.2])    return self.state
def step(self, action):    s = self.state
    # Recompensa según estado + acción (mundo real simplificado)    if
action == 1: # recomendar    if s == 2: reward = 1    # usuario
interesado → buena recomendación    elif s == 0: reward = -1 # usuario
desinteresado → molesto    else: reward = 0    else:    reward
= 0 # no recomendar es seguro, pero no aporta valor
    # Simulamos cambio de estado del usuario (comportamiento realista)
transition = {    0: [0, 1], # desinteresado → puede volverse neutral
1: [0, 1, 2], # neutral → podría interesarse o desinteresarse    2: [1,
2], # interesado → puede mantenerse o bajar a neutral    }    self.state
= np.random.choice(transition[s])
    done = False # este entorno no termina por episodio    return
self.state, reward, done, {}
# AGENTE: Q-Learning
env = RecommendationEnv()
n_states = 3n_actions = 2
Q = np.zeros((n_states, n_actions))
alpha = 0.5 # learning rategamma = 0.9 # descuento futuroepsilon = 0.2 #
exploraciónepisodes = 1000
reward_history = []

# ENTRENAMIENTO
for ep in range(episodes):    state = env.reset()    total_reward = 0
    for _ in range(20): # pasos por episodio    # Política epsilon-greedy
if random.random() < epsilon:    action = np.random.randint(n_actions)
else:    action = np.argmax(Q[state])
        next_state, reward, done, _ = env.step(action)
        # Actualización Q-Learning    td_target = reward + gamma *
np.max(Q[next_state])    Q[state, action] += alpha * (td_target - Q[state,
action])
        state = next_state    total_reward += reward
        reward_history.append(total_reward)

# RESULTADOS
print("Q-table aprendida:")print(Q.round(3))
print("\nPolicy aprendida (0=no recomendar, 1=recomendar):")best_actions =
np.argmax(Q, axis=1)for s, a in enumerate(best_actions):    labels =
["Desinteresado", "Neutral", "Interesado"]    print(f" Estado '{labels[s]}' →
Acción recomendada: {a}")

# PRUEBA DEL AGENTE TRAS EL APRENDIZAJE
print("\nSimulación final usando la policy aprendida:\n")state = env.reset()for t
in range(10):    action = np.argmax(Q[state])    next_state, reward, _, _ =
env.step(action)    print(f"Paso {t+1} | Estado={state} | Acción={action} |
Recompensa={reward} | Próximo={next_state}")    state = next_state

# Comentarios clave# - El estado representa el nivel de interés del usuario.# - La
acción es "recomendar" o "no recomendar".# - El agente aprende probando y
recibiendo recompensas.# - La Q-table captura el valor esperado de cada acción en
cada estado.# - La policy aprendida muestra "qué debería hacer" el agente en cada
estado.

```

Q-table aprendida:

[[2.202 0.945]

[2.348 2.238]

[3.494 4.591]]

Policy aprendida (0=no recomendar, 1=recomendar):

Estado 'Desinteresado' → Acción recomendada: 0

Estado 'Neutral' → Acción recomendada: 0

Estado 'Interesado' → Acción recomendada: 1

Simulación final usando la policy aprendida:

Paso 1 | Estado=1 | Acción=0 | Recompensa=0 | Próximo=1

Paso 2 | Estado=1 | Acción=0 | Recompensa=0 | Próximo=0

Paso 3 | Estado=0 | Acción=0 | Recompensa=0 | Próximo=0

Paso 4 | Estado=0 | Acción=0 | Recompensa=0 | Próximo=1

Paso 5 | Estado=1 | Acción=0 | Recompensa=0 | Próximo=0

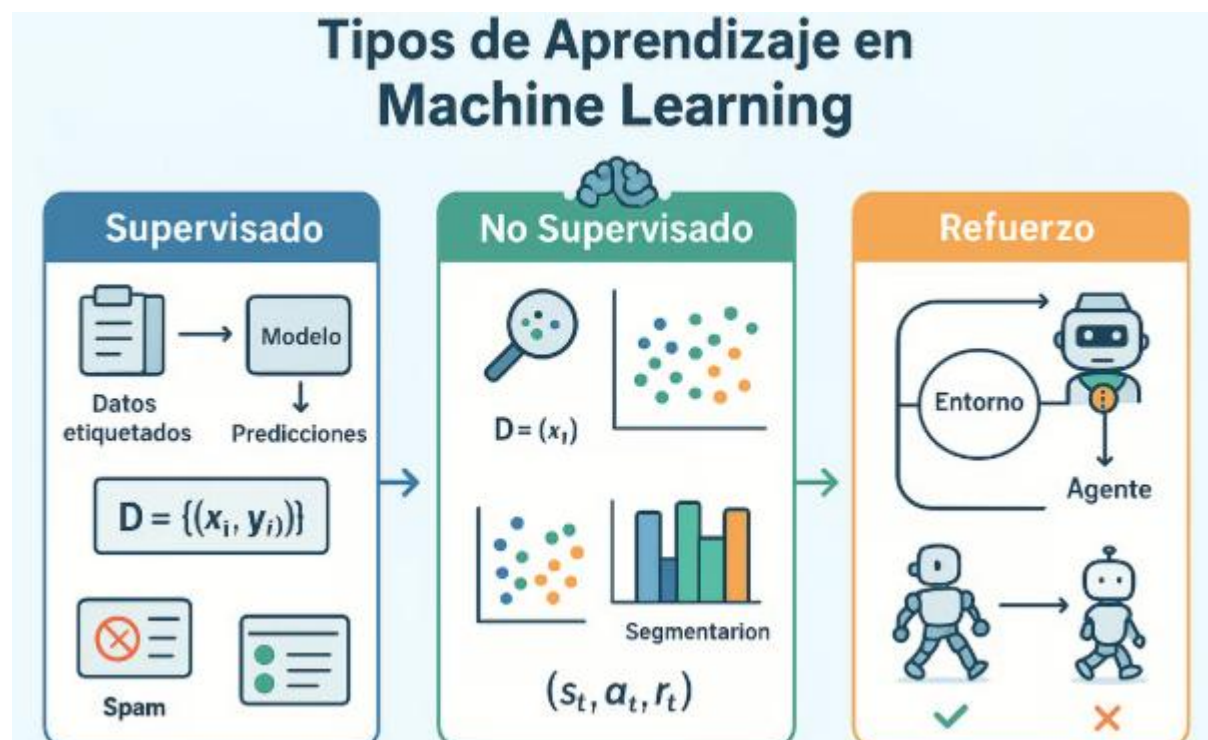
Paso 6 | Estado=0 | Acción=0 | Recompensa=0 | Próximo=1

Paso 7 | Estado=1 | Acción=0 | Recompensa=0 | Próximo=2

Paso 8 | Estado=2 | Acción=1 | Recompensa=1 | Próximo=2

Paso 9 | Estado=2 | Acción=1 | Recompensa=1 | Próximo=2

Paso 10 | Estado=2 | Acción=1 | Recompensa=1 | Próximo=2



Comparación rápida

Paradigma	Datos requeridos	Objetivo principal	Ejemplo práctico
Supervisado	Datos etiquetados	Predecir etiquetas o valores	Clasificación de imágenes
No supervisado	Datos sin etiquetas	Encontrar patrones o grupos	Segmentación de clientes
Por refuerzo	Interacción con entorno	Maximizar recompensa acumulada	Entrenamiento de agentes en juegos

Nota: Cada paradigma tiene sus ventajas y limitaciones, y la elección depende del problema y los datos disponibles.

En la industria de la inteligencia artificial, elegir el tipo correcto de aprendizaje es crucial para el éxito de un proyecto. Por ejemplo, si cuentas con un conjunto de datos bien etiquetado, el aprendizaje supervisado suele ser la mejor opción para tareas de predicción. Sin embargo, cuando los datos no están etiquetados o son costosos de etiquetar, el aprendizaje no supervisado permite descubrir insights valiosos sin supervisión directa.

El aprendizaje por refuerzo, aunque más complejo, es ideal para problemas donde las decisiones secuenciales y la interacción con un entorno dinámico son clave, como en robótica o sistemas de recomendación adaptativos.

Además, es importante considerar la calidad y cantidad de los datos disponibles, ya que esto influye directamente en el desempeño del modelo. La selección adecuada del tipo de aprendizaje también debe alinearse con los recursos computacionales accesibles y los tiempos de desarrollo previstos.

Finalmente, mantener una retroalimentación constante y realizar ajustes iterativos en el modelo contribuirá a optimizar los resultados obtenidos en cada etapa del desarrollo. En la industria de la inteligencia artificial, elegir el tipo correcto de aprendizaje es fundamental para garantizar el éxito y la eficiencia de un proyecto. Esta decisión depende en gran medida del tipo de datos con los que se cuenta y del objetivo que se desea alcanzar. Por ejemplo, si dispones de un conjunto de datos bien etiquetado, el aprendizaje supervisado suele ser la opción más adecuada para tareas de predicción y clasificación. Esta técnica implica entrenar un modelo con ejemplos concretos donde las entradas y sus correspondientes salidas están

claramente definidas, lo que facilita que el modelo aprenda a reconocer patrones y hacer predicciones precisas sobre nuevos datos.

Por otro lado, cuando los datos no están etiquetados o el proceso de etiquetado resulta costoso y laborioso, el aprendizaje no supervisado se convierte en una alternativa valiosa.

Este tipo de aprendizaje permite descubrir estructuras y patrones ocultos en los datos sin necesidad de supervisión directa, lo que es especialmente útil para tareas como agrupamiento (clustering) o reducción de dimensionalidad. Por ejemplo, puedes aplicar técnicas simples como el algoritmo k-means para agrupar datos similares, o usar el Análisis de Componentes Principales (PCA) para simplificar conjuntos de datos complejos manteniendo la mayor cantidad de información posible.

Además del aprendizaje supervisado y no supervisado, existe el aprendizaje por refuerzo, que aunque es más complejo, resulta ideal para problemas donde las decisiones secuenciales y la interacción con un entorno dinámico son fundamentales. En este contexto, un agente aprende a tomar acciones que maximicen una recompensa acumulada a lo largo del tiempo, lo cual es muy útil en campos como la robótica o los sistemas de recomendación adaptativos. Para comenzar con el aprendizaje por refuerzo, se pueden implementar técnicas sencillas como el método de Monte Carlo o los algoritmos de diferencia temporal (TD), que permiten familiarizarse con la idea de aprender a través de la experiencia directa y la retroalimentación.

Comprender estas diferencias y técnicas te permitirá diseñar soluciones más efectivas y alineadas con los objetivos y recursos disponibles para tu proyecto. También facilitará la elección del enfoque más apropiado según las características de los datos y la naturaleza del problema a resolver. Además, esta base te prepara para las próximas unidades, donde comenzarás a trabajar con datos reales y a crear modelos prácticos utilizando herramientas como Python y Jupyter Notebooks, aprendiendo a implementar desde técnicas básicas hasta métodos más avanzados de inteligencia artificial

5 - Validación: train/test, k-fold y selección de hiperparámetros

Ejercicios prácticos para aplicar validación cruzada, comparar modelos mediante pipelines reproducibles y automatizar la selección de hiperparámetros usando GridSearchCV y RandomizedSearchCV en Python.

Unidad: Validación: train/test, k-fold y selección de hiperparámetros

En esta unidad, pondremos en práctica la **validación cruzada** para evaluar modelos de Machine Learning, implementaremos **pipelines** para asegurar reproducibilidad y automatizaremos la **selección de hiperparámetros** usando `GridSearchCV` y `RandomizedSearchCV` de scikit-learn.

Conceptos clave

- **Validación cruzada k-fold:** Divide el dataset en k partes, entrena y valida el modelo k veces, cada vez con una parte diferente como conjunto de validación.
 - **Pipelines:** Encadenan pasos de preprocesamiento y modelado para evitar fugas de datos y facilitar la experimentación.
 - **GridSearchCV y RandomizedSearchCV:** Métodos para buscar automáticamente los mejores hiperparámetros evaluando combinaciones mediante validación cruzada.
-

Ejercicio práctico

Implementarás un pipeline que incluya preprocesamiento y un modelo de clasificación. Luego, aplicarás validación cruzada para evaluar el rendimiento y usarás `GridSearchCV` y `RandomizedSearchCV` para encontrar los mejores hiperparámetros.

Te recomendamos este dataset pero recordá que puedes usar el que elijas:

<https://www.kaggle.com/datasets/blastchar/telco-customer-churn>

Dataset Recomendado: Customer Churn

- **Regresión Logística (Clasificación):** La variable objetivo, Churn (abandono), es **binaria** (Sí/No), perfecta para entrenar un modelo de **regresión logística**. Esto te permitirá evaluar métricas de clasificación como precisión, *recall*, F1-score y la matriz de confusión.
- **Regresión Lineal (Regresión):** Aunque es un problema de clasificación, puedes usar variables numéricas continuas como MonthlyCharges (Cargos Mensuales) o TotalCharges (Cargos Totales) como variable objetivo, tratando de predecir su valor basándose en otras características, lo que te permite practicar la **regresión lineal** y sus métricas como MSE o R^2 .
- **Preprocesamiento:** Contiene una mezcla de **variables categóricas** (como tipo de contrato, servicios contratados, etc.) y **variables numéricas**, además de posibles **valores nulos** o inconsistencias (e.g., TotalCharges), lo que te obliga a construir *pipelines* robustos para:
 - Manejar datos categóricos (ej. OneHotEncoder).
 - Imputar valores faltantes.
 - Escalar datos numéricos (ej. StandardScaler).

Te pasamos el inicio del código para que arranques. Muchos éxitos!!

```
import pandas as pd
from sklearn.model_selection import train_test_split, KFold, GridSearchCV,
RandomizedSearchCV
from sklearn.preprocessing import OneHotEncoder, StandardScaler
from sklearn.compose import ColumnTransformer
from sklearn.pipeline import Pipeline
from sklearn.linear_model import LogisticRegression
# Cargar el dataset (ya disponible en el entorno)
df = pd.read_csv("/mnt/data/WA_Fn-UseC_-Telco-Customer-Churn.csv")
# Variable objetivo y = df["Churn"].map({"Yes": 1, "No": 0}) # Conversión a
binario
X = df.drop("Churn", axis=1)
# Identificar columnas numéricas y categóricas
num_cols = X.select_dtypes(include=["int64", "float64"]).columns
cat_cols = X.select_dtypes(include=["object"]).columns
# Preprocesamiento para cada tipo de variable
numeric_transformer = Pipeline(steps=[ # COMPLETAR: imputación y/o escalado])
categorical_transformer = Pipeline(steps=[ # COMPLETAR: imputación y
OneHotEncoder])
preprocessor = ColumnTransformer( transformers=[ ("num",
numeric_transformer, num_cols), ("cat", categorical_transformer, cat_cols)
])
# Pipeline completo (preprocesamiento + modelo)
clf = Pipeline(steps=[ ("preprocessor", preprocessor), ("model",
LogisticRegression()) # pueden cambiar el modelo si quieren])
# Definir un esquema de validación cruzada
kf = KFold(n_splits=5, shuffle=True, random_state=42)
# GridSearch y RandomizedSearch - Para que los estudiantes completen
param_grid = {
# COMPLETAR: hiperparámetros a buscar}
```

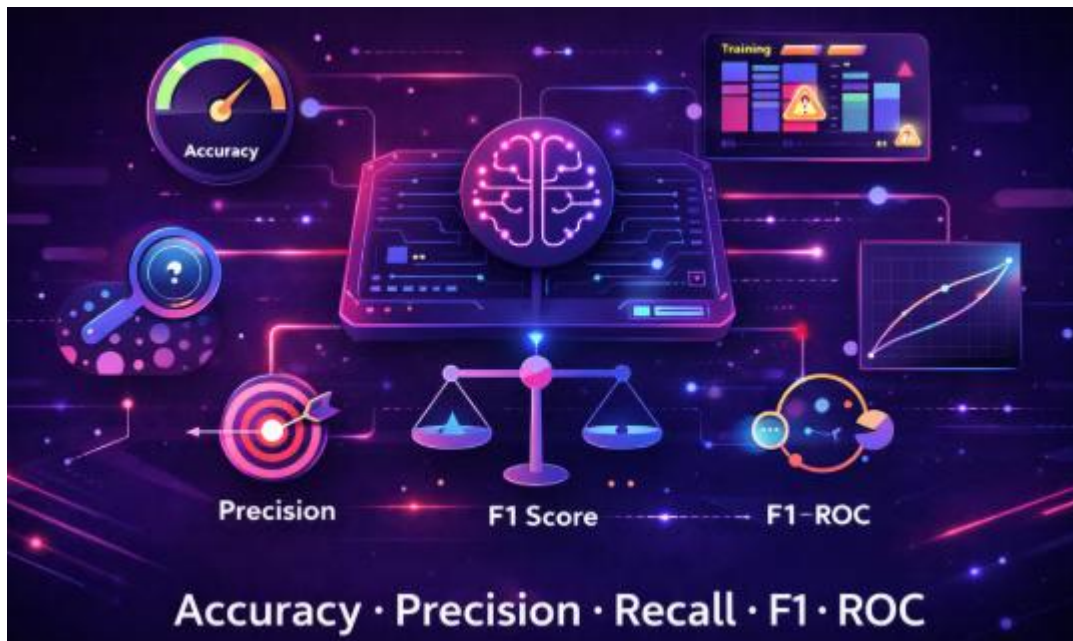
```
grid_search = GridSearchCV(    estimator=clf,    param_grid=param_grid,
cv=kfold,    scoring="f1",    n_jobs=-1)
# random_search = RandomizedSearchCV(
#     estimator=clf,
#     param_distributions={},
#     cv=kfold,
#     scoring="f1",
#     n_iter=20,
#     n_jobs=-1
# )
```

6 - Métricas y diagnóstico: accuracy, precision, recall, F1, ROC

Introducción clara y práctica a las métricas clave para evaluar modelos de clasificación, con énfasis en cuándo y cómo elegir cada métrica según el contexto, y revisión de sobreajuste y técnicas para mitigarlo.

¿Alguna vez te has preguntado cómo saber si un modelo de Machine Learning está funcionando bien? No basta con que prediga algo; necesitamos medir su desempeño con métricas que reflejen la realidad del problema. En esta unidad, exploraremos las métricas más comunes para evaluar modelos de clasificación, como *accuracy*, *precision*, *recall*, *F1* y *ROC-AUC*. Aprenderás cuándo es mejor usar cada una, especialmente en casos donde las clases están desequilibradas, un escenario muy común en la industria. Además, repasaremos el concepto de *sobreajuste* (overfitting), un problema frecuente que puede hacer que un modelo parezca excelente en datos de entrenamiento pero falle en datos nuevos. Veremos técnicas prácticas para mitigarlo, reforzando lo aprendido previamente.

Al finalizar, serás capaz de seleccionar la métrica adecuada para tu proyecto y diagnosticar errores comunes en la evaluación, habilidades esenciales para cualquier profesional que trabaje con modelos de Machine Learning.



Métricas de Evaluación en Machine Learning

Para evaluar un modelo de clasificación, existen varias métricas que nos ayudan a entender su desempeño desde diferentes perspectivas. A continuación, definimos las principales:

Accuracy (Exactitud)

Es la proporción de predicciones correctas sobre el total de casos:

$$\text{Accuracy} = \frac{TP + TN}{TP + TN + FP + FN}$$

- *TP*: Verdaderos positivos
- *TN*: Verdaderos negativos
- *FP*: Falsos positivos
- *FN*: Falsos negativos

Limitación: En datasets desequilibrados, un modelo que siempre predice la clase mayoritaria puede tener alta accuracy pero ser inútil.

Precision (Precisión)

Mide la proporción de predicciones positivas correctas sobre todas las predicciones positivas:

$$\text{Precision} = \frac{TP}{TP + FP}$$

Es útil cuando el costo de falsos positivos es alto, por ejemplo, en detección de fraude.

Recall (Sensibilidad o Exhaustividad)

Indica la proporción de casos positivos correctamente identificados:

$$\text{Recall} = \frac{TP}{TP + FN} \quad \boxed{\text{Recall} = \frac{TP}{TP + FN}}$$

Es clave cuando es crítico detectar todos los casos positivos, como en diagnósticos médicos.

F1 Score

Es la media armónica entre precision y recall, balanceando ambos:

$$F1 = \frac{2 \times \text{Precision} \times \text{Recall}}{\text{Precision} + \text{Recall}} \quad \boxed{F1 = \frac{2 \times \text{Precision} \times \text{Recall}}{\text{Precision} + \text{Recall}}}$$

Ideal cuando se busca un equilibrio entre precision y recall.

ROC y AUC

La curva ROC (Receiver Operating Characteristic) muestra la relación entre la tasa de verdaderos positivos y la tasa de falsos positivos para distintos umbrales. El área bajo la curva (AUC) resume esta relación en un solo valor entre 0 y 1.

- Un AUC cercano a 1 indica un modelo con buena capacidad discriminativa.

Sobreajuste y Regularización

El *sobreajuste* ocurre cuando un modelo aprende demasiado bien los detalles y ruido del conjunto de entrenamiento, perdiendo capacidad de generalización.

Técnicas comunes para mitigarlo:

- **Regularización L1 y L2:** Añaden penalizaciones a los coeficientes del modelo para evitar valores extremos.
- **Pruning:** En modelos como árboles de decisión, se eliminan ramas que aportan poco valor.
- **Early stopping:** Detener el entrenamiento antes de que el modelo se ajuste demasiado.

Estas técnicas ayudan a crear modelos más robustos y confiables en producción.

En la industria, elegir la métrica correcta puede marcar la diferencia entre un modelo útil y uno que cause pérdidas o riesgos. Por ejemplo, en detección de fraudes bancarios, un falso positivo (bloquear una transacción legítima) puede molestar al cliente, pero un falso negativo (no detectar fraude) puede causar pérdidas económicas.

Por eso, entender y aplicar correctamente métricas como precision y recall es fundamental. Además, el sobreajuste es un enemigo silencioso que puede hacer que un modelo funcione bien en pruebas pero falle en producción, afectando la confianza en soluciones de IA.

Esta unidad te prepara para diagnosticar estos aspectos y tomar decisiones informadas en tus proyectos de Machine Learning, usando Python y herramientas como scikit-learn para calcular estas métricas y aplicar técnicas de regularización.

7 - Reproducibilidad y despliegue mínimo viable

Introducción a las buenas prácticas para guardar modelos, controlar experimentos y preparar resultados para producción, enfocándose en la reproducibilidad y el despliegue básico en proyectos de Machine Learning.

¿Alguna vez has entrenado un modelo de Machine Learning y luego no has podido replicar exactamente los mismos resultados? Este es un problema común en proyectos reales y puede afectar la confianza en tus modelos y la colaboración con otros equipos. En esta unidad, exploraremos cómo garantizar que tus experimentos sean reproducibles y cómo preparar tus modelos para un despliegue mínimo viable, es decir, una versión funcional y básica que pueda ser usada en producción o presentada a stakeholders.

A lo largo de esta sesión, aprenderás prácticas clave para guardar modelos de forma segura, controlar versiones de tus notebooks y resultados, y recomendaciones para que tus proyectos sean transparentes y confiables. Estas habilidades son esenciales para cualquier profesional que quiera llevar sus proyectos de IA y ML más allá del laboratorio y hacia aplicaciones reales, manteniendo la ética y la calidad en el proceso.



Reproducibilidad y despliegue mínimo viable en Machine Learning

¿Qué es la reproducibilidad en ML?

La reproducibilidad se refiere a la capacidad de repetir un experimento o entrenamiento de modelo y obtener resultados consistentes. Esto es fundamental para validar hallazgos, colaborar con otros y asegurar la calidad del producto final.

***Nota:** La reproducibilidad no solo implica guardar el modelo, sino también documentar y controlar el entorno, los datos y el código usados.*

Serialización y almacenamiento de modelos

Para guardar un modelo entrenado y poder reutilizarlo sin necesidad de reentrenar, se utiliza la **serialización**. En Python, bibliotecas como `pickle` o las funciones específicas de `scikit-learn` permiten guardar modelos en archivos.

```
python
import joblib
# Guardar modelo
joblib.dump(modelo_entrenado, 'modelo.pkl')
# Cargar modelo
modelo_cargado = joblib.load('modelo.pkl')
```

Buenas prácticas:

- Guardar versiones con nombres que incluyan fecha o versión.

- Documentar parámetros y métricas asociadas a cada modelo.

Control de versiones para notebooks y experimentos

El código y los notebooks deben estar bajo control de versiones, típicamente usando Git. Esto permite:

- Rastrear cambios y revertir errores.
- Colaborar con otros desarrolladores.
- Mantener un historial claro de experimentos.

Además, herramientas como MLflow o Weights & Biases facilitan el seguimiento de experimentos, métricas y artefactos.

Recomendaciones para reproducibilidad y despliegue mínimo viable

- **Documenta tu entorno:** versiones de librerías, sistema operativo, hardware.
- **Usa entornos virtuales:** como venv o conda para aislar dependencias.
- **Automatiza la preparación de datos y entrenamiento:** scripts claros y parametrizados.
- **Prepara un paquete mínimo:** que incluya modelo serializado, código para inferencia y documentación.
- **Comunica resultados con claridad:** incluye métricas, limitaciones y contexto para stakeholders.

Ejemplo de industria: Una startup de salud que desarrolla un modelo para detectar enfermedades debe garantizar que su modelo pueda ser reproducido y auditado por reguladores, asegurando confianza y cumplimiento ético.

En proyectos reales de Machine Learning, la reproducibilidad es clave para evitar sorpresas y errores costosos. Por ejemplo, si un equipo de datos entrega un modelo a producción sin documentación ni control de versiones, puede ser difícil replicar resultados o corregir fallos.

Además, el despliegue mínimo viable permite entregar una versión funcional del modelo que puede ser evaluada y mejorada iterativamente, facilitando la comunicación con stakeholders y la integración en sistemas existentes.

Esta unidad conecta con conceptos previos sobre limpieza de datos y evaluación de modelos, y prepara el terreno para la siguiente unidad, donde se abordarán buenas prácticas éticas y comunicación efectiva de resultados.

8 - Introducción a casos de uso y ejemplos reales

Explora cómo los diferentes tipos de problemas en la industria se abordan mediante técnicas de Machine Learning y Deep Learning, con ejemplos prácticos de clasificación, regresión, clustering, reconocimiento y sistemas de recomendación.

¿Alguna vez te has preguntado cómo plataformas como Netflix o Amazon saben qué recomendarte, o cómo los bancos detectan transacciones fraudulentas? Estos son ejemplos reales de cómo la Inteligencia Artificial (IA) y el Machine Learning (ML) están transformando industrias enteras. En esta unidad, exploraremos casos de uso comunes en la industria que ilustran cómo diferentes tipos de problemas se abordan con técnicas de ML y Deep Learning. Comprenderás conceptos clave como clasificación, regresión, clustering, reconocimiento y sistemas de recomendación, y cómo se aplican en contextos reales. Este conocimiento te permitirá relacionar problemas concretos con las soluciones adecuadas, un paso fundamental para avanzar en tu aprendizaje de IA y ML.

Casos de uso comunes en IA y ML

Clasificación

La **clasificación** es un problema de ML donde el objetivo es asignar una etiqueta o categoría a un dato de entrada. Por ejemplo, en la industria financiera, los bancos usan clasificación para detectar transacciones fraudulentas, etiquetando cada transacción como "fraudulenta" o "legítima".

Ejemplo:

Clasificación: Detección de fraude

Objetivo: mostrar paso a paso un flujo sencillo de clasificación supervisada aplicado a detección de fraude.

Generamos un dataset sintético desbalanceado (1% fraude) para ilustrar problemas reales: métricas a priorizar (precision/recall), tratamiento del desbalanceo y evaluación.

Estructura:

1. Imports
2. Generación de datos sintéticos (simulando fraude)

3. EDA rápido (balance de clases)
4. División train/test y definición de pipelines
5. Entrenamiento de modelos (Logistic Regression y Random Forest)
6. Evaluación: accuracy, precision, recall, f1, AUC, matriz de confusión.

Video:

```
# Imports ---%matplotlib inline
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
from sklearn.datasets import make_classification
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.pipeline import make_pipeline
from sklearn.linear_model import LogisticRegression
from sklearn.ensemble import RandomForestClassifier
from sklearn.metrics import (
    accuracy_score, precision_score, recall_score, f1_score,
    roc_auc_score, confusion_matrix, classification_report,
    roc_curve)
import joblib # para guardar modelos si querés
# Generación de datos sintéticos (clase minoritaria ~1% para simular
fraude) ---RANDOM_STATE = 42
X, y = make_classification(n_samples=10000, n_features=20,
n_informative=5, n_redundant=2, n_repeated=0, n_classes=2,
weights=[0.99, 0.01], # 1% de la clase positiva -> fraude flip_y=0.01,
class_sep=1.0, random_state=RANDOM_STATE)
#-----
# convertir a DataFrame para comodidad
feature_names = [f"f{i}" for i in range(X.shape[1])]
df = pd.DataFrame(X, columns=feature_names)
df['is_fraud'] = y
# mostrar primeras filas y balance de clases
display(df.head())
print("\nBalance de clases (train):")
print(df['is_fraud'].value_counts(normalize=False))
print("\nProporción de fraude:")
print(df['is_fraud'].value_counts(normalize=True))
#-----
Salida:
Balance de clases (train):is_fraud0    98431    157Name: count, dtype: int64
Proporción de fraude:is_fraud0    0.98431    0.0157Name: proportion, dtype:
float64
#-----
#EDA rápido: distribución de clases
counts = df['is_fraud'].value_counts().sort_index()
plt.figure(figsize=(6,4))
plt.bar(['Legítima (0)', 'Fraude (1)'], counts.values)
plt.title('Distribución de clases (ejemplo sintético)')
plt.ylabel('Cantidad de muestras')
for i, v in enumerate(counts.values):
    plt.text(i, v + max(counts.values)*0.01, str(v), ha='center')
```

```

plt.show()
# --- 4) División train/test (estratificada) ---X =
df[feature_names].valuesy = df['is_fraud'].values
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.25,
stratify=y, random_state=RANDOM_STATE)
print("Tamaños: X_train:", X_train.shape, "X_test:", X_test.shape)
print("Balance en train:", np.bincount(y_train), " balance en test:",
np.bincount(y_test))
#-----
-----
Salida:
Tamaños: X_train: (7500, 20) X_test: (2500, 20)Balance en train: [7382  118]
balance en test: [2461   39]
#-----
-----
# Definición de modelos / pipelines # Logistic Regression con escalado - uso
class_weight='balanced' para mitigar el desbalanceolog_pipe = make_pipeline(
StandardScaler(), LogisticRegression(solver='liblinear',
class_weight='balanced', random_state=RANDOM_STATE))
# Random Forest (no necesita escalado)
rf_clf = RandomForestClassifier(n_estimators=200, class_weight='balanced',
random_state=RANDOM_STATE, n_jobs=-1)
# Entrenamiento
print("Entrenando Logistic Regression...")log_pipe.fit(X_train, y_train)
print("Entrenando Random Forest...")rf_clf.fit(X_train, y_train)
#-----
-----
#Evaluación: función auxiliar y evaluación de ambos modelos ---
def evaluate_model(model, X_test, y_test, model_name="model"):
    if hasattr(model, "predict_proba"):
        y_prob = model.predict_proba(X_test)[:, 1]
    else:
        # si no tiene predict_proba, usar decision_function si existe
    if hasattr(model, "decision_function"):
        y_prob = model.decision_function(X_test)
    # no es estrictamente probabilidad, pero sirve para ROC
    else:
        # fallback: usar predict (no ideal para AUC)
y_prob = model.predict(X_test)
    y_pred = model.predict(X_test)
    acc = accuracy_score(y_test, y_pred)
    prec = precision_score(y_test, y_pred, zero_division=0)
    rec = recall_score(y_test, y_pred, zero_division=0)
    f1 = f1_score(y_test, y_pred, zero_division=0)
    try:
        auc = roc_auc_score(y_test, y_prob)
    except Exception:
        auc = float("nan")
    print(f"--- Evaluación: {model_name} ---")    print(f"Accuracy: {acc:.4f}")
print(f"Precision: {prec:.4f}")    print(f"Recall: {rec:.4f}")    print(f"F1-
score: {f1:.4f}")    print(f"AUC: {auc:.4f}")    print("\nClassification report:")
print(classification_report(y_test, y_pred, digits=4, zero_division=0))
    # matriz de confusión
    cm = confusion_matrix(y_test, y_pred)
    plt.figure(figsize=(4,3))
    plt.imshow(cm, interpolation='nearest', cmap=plt.cm.Blues)
    plt.title(f'Confusión: {model_name}')
    plt.colorbar()
    ticks = [0,1]    plt.xticks(ticks, ['Legítima (0)', 'Fraude (1)'])
    plt.yticks(ticks, ['Legítima (0)', 'Fraude (1)'])

```

```

    thresh = cm.max() / 2.    for i in range(cm.shape[0]):
        for j in range(cm.shape[1]):
            plt.text(j, i, format(cm[i, j], 'd'),
                ha="center", va="center",
                color="white" if cm[i, j] > thresh else "black")
plt.ylabel('Etiqueta real')
plt.xlabel('Etiqueta predicha')
plt.show()
return {'accuracy': acc, 'precision': prec, 'recall': rec, 'f1': f1, 'auc':
auc}
# Evaluar Logistic Regressionres_log = evaluate_model(log_pipe, X_test, y_test,
model_name="Logistic Regression")
# Evaluar Random Forestres_rf = evaluate_model(rf_clf, X_test, y_test,
model_name="Random Forest")
#-----
-----
Salida:
--- Evaluación: Logistic Regression ---Accuracy: 0.6880Precision: 0.0256Recall:
0.5128F1-score: 0.0488AUC: 0.5944
Classification report:
              precision    recall  f1-score   support

0           0.9889       0.6908       0.8134         2461
0.5128    0.0488         39
              accuracy              0.6880         2500  macro avg       0.5073
0.6018    0.4311         2500weighted avg       0.9739       0.6880       0.8015         2500
--- Evaluación: Random Forest ---Accuracy: 0.9844Precision: 0.0000Recall:
0.0000F1-score: 0.0000AUC: 0.8181
Classification report:
              precision    recall  f1-score   support

0           0.9844       1.0000       0.9921         2461
0.0000    0.0000         39
              accuracy              0.9844         2500  macro avg       0.4922
0.5000    0.4961         2500weighted avg       0.9690       0.9844       0.9767         2500

```

Regresión

La **regresión** busca predecir un valor numérico continuo a partir de datos de entrada. Un caso típico es la predicción del precio de una vivienda basado en características como tamaño, ubicación y número de habitaciones.

Ejemplo:

```

#
=====
# MODELO DE REGRESIÓN – PREDICCIÓN DE PRECIOS DE VIVIENDAS #
=====
# -----
# 1) Imports
# -----
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
from sklearn.model_selection import train_test_split, GridSearchCV
from sklearn.preprocessing import StandardScaler, OneHotEncoder

```

```

from sklearn.compose import ColumnTransformer
from sklearn.pipeline import make_pipeline
from sklearn.linear_model import LinearRegression, Ridge
from sklearn.ensemble import RandomForestRegressor
from sklearn.metrics import mean_absolute_error, mean_squared_error,
r2_score
import joblib

# -----
# 2) Generación de datos sintéticos
# -----
# Simulamos un dataset inmobiliario con superficie, ambientes, baños,
antigüedad,
# distancia al centro, barrio y precio.
np.random.seed(42)
n = 4000
area_m2 = np.random.normal(80, 30, n).clip(20, 400)
rooms = np.random.choice([1,2,3,4,5], n, p=[0.05,0.35,0.35,0.2,0.05])
bathrooms = np.clip(rooms + np.random.choice([0,0,1], n), 1, 4)
age = np.random.exponential(15, n).astype(int)
dist_center_km = np.abs(np.random.normal(8, 5, n)).clip(0.2, 40)
neighs = ['A','B','C','D','E']
neigh = np.random.choice(neighs, n, p=[0.12,0.25,0.3,0.2,0.13])
neigh_eff = {'A':1.25,'B':1.05,'C':1.0,'D':0.9,'E':0.8}
base_m2 = 1200
price = (    base_m2 * area_m2 * np.array([neigh_eff[x] for x in neigh])    +
rooms * 15000    + bathrooms * 8000    - age * 300    - dist_center_km * 2000    +
np.random.normal(0, 30000, n))
price = np.maximum(price, 10000)
df = pd.DataFrame({'area_m2': np.round(area_m2,1),    'rooms': rooms,
'bathrooms': bathrooms,    'age': age,    'dist_center_km':
np.round(dist_center_km,2),    'neighborhood': neigh,    'price':
np.round(price,2)})
print(df.head())
# -----
# 3) División train/test
# -----
X = df.drop(columns=['price'])
y = df['price']
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.25,
random_state=42, stratify=X['neighborhood'])
print("Tamaños:", X_train.shape, X_test.shape)
# -----
# 4) Preprocesamiento con ColumnTransformer
# -----
numeric_features = ['area_m2','rooms','bathrooms','age','dist_center_km']
categorical_features = ['neighborhood']
# FIX: usar sparse_output=False en versiones nuevas de
sklearn
categorical_transformer = OneHotEncoder(handle_unknown='ignore',
sparse_output=False)
numeric_transformer = StandardScaler()
preprocessor = ColumnTransformer([    ('num', numeric_transformer,
numeric_features),    ('cat', categorical_transformer, categorical_features)])
# -----
# 5) Modelos a evaluar
# -----
pipe_lr = make_pipeline(preprocessor, LinearRegression())
pipe_ridge = make_pipeline(preprocessor, Ridge(alpha=1.0))
pipe_rf = make_pipeline(preprocessor, RandomForestRegressor(
n_estimators=200, random_state=42, n_jobs=-1))

```

```

# Entrenamiento
pipe_lr.fit(X_train, y_train)
pipe_ridge.fit(X_train, y_train)
pipe_rf.fit(X_train, y_train)
# -----
# 6) Función de evaluación
# -----
def evaluar(model, X_test, y_test, nombre):
    y_pred = model.predict(X_test)
    mae = mean_absolute_error(y_test, y_pred)
    mse = mean_squared_error(y_test, y_pred)
    rmse = np.sqrt(mse)
    r2 = r2_score(y_test, y_pred)
    print(f"\n--- {nombre} ---")
    print(f"MAE: {mae:,.2f}")
    print(f"MSE: {mse:,.2f}")
    print(f"RMSE: {rmse:,.2f}")
    print(f"R²: {r2:.4f}")
    return y_pred
# Evaluación
pred_lr = evaluar(pipe_lr, X_test, y_test, "Linear Regression")
pred_ridge = evaluar(pipe_ridge, X_test, y_test, "Ridge")
pred_rf = evaluar(pipe_rf, X_test, y_test, "Random Forest")
# -----
# 7) Gráfico real vs predicho (RF)
# -----
plt.figure(figsize=(6,6))
plt.scatter(y_test, pred_rf, alpha=0.3)
plt.plot([y_test.min(), y_test.max()], [y_test.min(), y_test.max()], "k--")
plt.xlabel("Precio real")
plt.ylabel("Precio predicho")
plt.title("Random Forest - Real vs Predicho")
plt.show()
# -----
# 8) Importancia de features (RF)
# -----
# Extraer nombres transformados
ohe = pipe_rf.named_steps["columntransformer"].named_transformers_["cat"]
ohe_names = ohe.get_feature_names_out(categorical_features)
feature_names = numeric_features + list(ohe_names)
importances = pipe_rf.named_steps["randomforestregressor"].feature_importances_
feat_imp = pd.Series(importances, index=feature_names).sort_values(ascending=False)
print("\nImportancia de Features (RF):")
print(feat_imp)
plt.figure(figsize=(8,4))
feat_imp.plot(kind="bar")
plt.title("Importancia de Features - Random Forest")
plt.show()

```

area_m2 rooms bathrooms age dist_center_km neighborhood price

0 94.9 3 3 10 10.23 E 78229.90

1 75.9 2 3 17 0.94 E 138341.29

2 99.4 5 4 1 10.33 C 226575.92

3 125.7 3 4 12 2.37 C 202232.54

4 73.0 3 4 4 12.71 C 137013.29

Tamaños: (3000, 6) (1000, 6)

--- Linear Regression ---

MAE: 25,229.01

MSE: 983,436,352.90

RMSE: 31,359.79

R²: 0.6738

--- Ridge ---

MAE: 25,228.97

MSE: 983,436,701.17

RMSE: 31,359.79

R²: 0.6738

--- Random Forest ---

MAE: 27,074.90

MSE: 1,130,568,489.76

RMSE: 33,623.93

R²: 0.6250

Importancia de Features (RF):

area_m2 0.540528

dist_center_km 0.136453

rooms 0.099778

age 0.081064

bathrooms 0.055101

neighborhood_A 0.038002

neighborhood_E 0.019601

neighborhood_D 0.010938

neighborhood_B 0.009380

neighborhood_C 0.009154

dtype: float64

Clustering

El **clustering** agrupa datos similares sin etiquetas previas. En marketing, se usa para segmentar clientes en grupos con comportamientos similares, facilitando campañas personalizadas.

Ejemplo:

```
#  
=====
```

```

==
# CLUSTERING DIDÁCTICO CON KMEANS – SEGMENTACIÓN DE CLIENTES #
=====
==
# -----
# 1) Imports y configuración
# -----
%matplotlib inline
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
from sklearn.preprocessing import StandardScaler
from sklearn.decomposition import PCA
from sklearn.cluster import KMeans
from sklearn.metrics import silhouette_score
import joblib
RND = 42 # semilla para reproducibilidad
# -----
# 2) Generación de datos sintéticos tipo RFM + comportamiento
# -----
# Queremos simular clientes con características típicas para segmentación:
# - annual spend: gasto anual
# - transactions_per_month: cantidad de compras por mes
# - avg_order_value: ticket promedio
# - tenure_months: antigüedad en meses
# - visits_last_month: visitas al canal/plataforma el último mes
np.random.seed(RND)
n_customers = 1200
# Generamos 3 grupos "reales" subyacentes y mezclamos
centers = [ # altas
compras, frecuentes, alto ticket, larga antigüedad, muchas visitas [15000, 12,
1250, 60, 8], # moderados [6000, 4, 900, 24, 3], # bajos [1200, 1,
300, 6, 1]]
cluster_sizes = [0.35, 0.45, 0.20]
X_parts = []
labels_true = []
for i, (c, frac) in enumerate(zip(centers, cluster_sizes)):
    size = int(n_customers * frac)
    # añadir ruido gaussiano alrededor de cada centro
    part = np.random.normal(loc=c, scale=[2000, 2, 200, 10, 1], size=(size,
5))
    # asegurar valores plausibles (no negativos)
    part[:,0] = np.abs(part[:,0])
    # annual_spend part[:,1] = np.clip(part[:,1], 0, 60) #
    transactions_per_month part[:,2] = np.abs(part[:,2])
    # avg_order_value part[:,3] = np.clip(part[:,3], 0, 600) # tenure_months
    part[:,4] = np.clip(np.round(part[:,4]), 0, 30) # visits_last_month
    X_parts.append(part)
    labels_true += [i] * size
X = np.vstack(X_parts)
# Si quedaron menos por redondeo, completar con ruido alrededor del centro
if X.shape[0] < n_customers:
    extra = n_customers - X.shape[0]
    extra_part =
np.random.normal(loc=centers[1], scale=[2000,2,200,10,1], size=(extra,5))
    extra_part[:,0] = np.abs(extra_part[:,0])
    extra_part[:,1] = np.clip(extra_part[:,1], 0, 60)
    extra_part[:,2] = np.abs(extra_part[:,2])
    extra_part[:,3] = np.clip(extra_part[:,3], 0, 600)
    extra_part[:,4] = np.clip(np.round(extra_part[:,4]), 0, 30)
    X =
np.vstack([X, extra_part])
    labels_true += [1]*extra

```

```

# Construir DataFrame con nombres de columnas
cols = ['annual_spend', 'transactions_per_month', 'avg_order_value', 'tenure_months', 'visits_last_month']
df = pd.DataFrame(X, columns=cols).reset_index(drop=True)
df = df.round({'annual_spend':2, 'avg_order_value':2, 'transactions_per_month':0, 'tenure_months':0, 'visits_last_month':0})
df['transactions_per_month'] = df['transactions_per_month'].astype(int)
df['tenure_months'] = df['tenure_months'].astype(int)
df['visits_last_month'] = df['visits_last_month'].astype(int)
print("Dataset simulado – primeras filas:")
display(df.head())

# -----
# 3) EDA simple: estadísticos y distribución
# -----
print("\nEstadísticas descriptivas:")
display(df.describe().T)
plt.figure(figsize=(10,3))
plt.subplot(1,3,1)
plt.hist(df['annual_spend'], bins=30)
plt.title('Distribución gasto anual')
plt.subplot(1,3,2)
plt.hist(df['transactions_per_month'], bins=15)
plt.title('Transacciones/mes')
plt.subplot(1,3,3)
plt.hist(df['avg_order_value'], bins=30)
plt.title('Ticket promedio')
plt.tight_layout()
plt.show()

# -----
# 4) Escalado (KMeans funciona mejor con variables escaladas)
# -----
scaler = StandardScaler()
X_scaled = scaler.fit_transform(df.values)

# -----
# 5) Buscar número de clusters: Elbow (inertia) + Silhouette
# - calculamos inertia para k=1..8
# - calculamos silhouette para k=2..8 y elegimos k que maximiza silhouette
# -----
inertias = []
sil_scores = []
K_range = range(1,9)
for k in K_range:
    km = KMeans(n_clusters=k, random_state=RND, n_init=10)
    km.fit(X_scaled)
    inertias.append(km.inertia_)
    if k >= 2:
        sil = silhouette_score(X_scaled, km.labels_)
        sil_scores.append(sil)
    else:
        sil_scores.append(np.nan)

# Plot Elbow + Silhouette
plt.figure(figsize=(10,4))
plt.subplot(1,2,1)
plt.plot(K_range, inertias, '-o')
plt.xlabel('k')
plt.ylabel('Inertia')
plt.title('Elbow method (Inertia)')
plt.subplot(1,2,2)
plt.plot([k for k in K_range if k>=2], sil_scores[1:], '-o')

```



```

plt.xlabel('k')
plt.ylabel('Silhouette score')
plt.title('Silhouette score (k>=2)')
plt.tight_layout()
plt.show()
# Elegir k con mejor silhouette
valid_ks = [k for k in K_range if k>=2]
best_k = valid_ks[int(np.nanargmax(sil_scores[1:]))] # índice desplazado
por k>=2
print(f"\nSilhouette scores (k=2..8): {np.round(sil_scores[1:],3)}")
print(f"Elegimos k = {best_k} (mejor silhouette)")
# -----
# 6) Entrenar KMeans final y asignar clusters
# -----
kmeans = KMeans(n_clusters=best_k, random_state=RND,
n_init=10).fit(X_scaled)
labels = kmeans.labels_.df['cluster'] = labels
print("\nConteo por cluster:")
print(df['cluster'].value_counts().sort_index())
# -----
# 7) Interpretación: centros en escala original (invirtiendo el escalado)
# -----
centers_scaled = kmeans.cluster_centers_
centers_original = scaler.inverse_transform(centers_scaled)
centers_df = pd.DataFrame(centers_original,
columns=cols)
centers_df.index.name = 'cluster'
print("\nCentros de los clusters (en escala original - interpretables):")
display(centers_df.round(2))
# -----
# 8) Perfilado de clusters: medias por cluster y tamaño
# -----
profile =
df.groupby('cluster')[cols].agg(['count','mean','median','std']).round(2)
print("\nPerfil resumido por cluster:")
display(profile)
# -----
# 9) Visualización 2D con PCA (para mostrar separación de clusters)
# -----
pca = PCA(n_components=2, random_state=RND)
X_pca = pca.fit_transform(X_scaled)
plt.figure(figsize=(7,5))
for cl in sorted(df['cluster'].unique()):
    mask = df['cluster'] == cl
    plt.scatter(X_pca[mask,0], X_pca[mask,1],
label=f'Cluster {cl}', alpha=0.6, s=30)
plt.xlabel('PCA 1')
plt.ylabel('PCA 2')
plt.title(f'Visualización PCA (k={best_k})')
plt.legend()
plt.grid(alpha=0.2)
plt.show()
# -----
# 10) Ejemplo de uso: asignar nuevo cliente a un cluster
# -----
nuevo_cliente = pd.DataFrame([
    'annual_spend': 8000,
    'transactions_per_month': 5,
    'avg_order_value': 1600,
    'tenure_months': 18,
    'visits_last_month': 4])
nuevo_scaled = scaler.transform(nuevo_cliente.values)
asignado = kmeans.predict(nuevo_scaled)[0]
print(f"\nNuevo cliente asignado al cluster: {asignado}")

```

```

print("Centro del cluster asignado (valores promedio):")
display(centers_df.loc[asignado].round(2))

# -----
# 11) Conclusión # -----
print("\nConclusiones rápidas:")
print("- Usamos KMeans para segmentar clientes sintéticos (RFM-like + comportamiento).")
print("- Seleccionamos k por silhouette automáticamente; conviene revisar elbow + negocio.")
print("- Interpretar centros en la escala original permite diseñar acciones por segmento.")
print("- Pasos siguientes: validar estabilidad (bootstrap), aplicar a datos reales, probar algoritmos jerárquicos o DBSCAN, y combinar con análisis de valor (LTV).")
Dataset simulado – primeras filas:

```

	annual_spend	transactions_per_month	avg_order_value	tenure_months	visits_last
0	15993.43	12	1379.54	75	8
1	14531.73	15	1403.49	55	9
2	14073.16	11	1298.39	41	6
3	13875.42	10	1312.85	51	7
4	17931.30	12	1263.51	46	7

Estadísticas descriptivas:

	count	mean	std	min	25%	50%
annual_spend	1200.0	8322.323217	5473.082152	0.36	3905.255	6950.60
transactions_per_month	1200.0	6.255833	4.750880	0.00	2.000	5.00
avg_order_value	1200.0	904.759742	390.868927	0.77	647.545	962.96

tenure_months	1200.0	33.155000	22.908732	0.00	14.000	28.50
visits_last_month	1200.0	4.402500	2.992985	0.00	2.000	3.00

Conteo por cluster: cluster 0 780 1 420
 Name: count, dtype: int64

Centros de los clusters (en escala original — interpretables):

	annual_spend	transactions_per_month	avg_order_value	tenure_months	visits
cluster					
0	4729.53	3.17	714.03	18.49	2.40
1	14994.66	11.99	1258.98	60.39	8.12

Perfil resumido por cluster:

	annual_spend				transactions_per_month				avg_ord
	count	mean	median	std	count	mean	median	std	count
cluster									
0	780	4729.53	4920.55	2684.37	780	3.17	3.0	2.23	780
1	420	14994.66	15048.73	1914.30	420	11.99	12.0	2.15	420

Nuevo cliente asignado al cluster: 0
 Centro del cluster asignado (valores promedio):

	0
--	----------

annual_spend	4729.53
transactions_per_month	3.17
avg_order_value	714.03
tenure_months	18.49
visits_last_month	2.40

dtype: float64

Conclusiones:

- Usamos KMeans para segmentar clientes sintéticos (RFM-like + comportamiento).-
- Seleccionamos k por silhouette automáticamente; conviene revisar elbow + negocio.-
- Interpretar centros en la escala original permite diseñar acciones por segmento.-
- Pasos siguientes: validar estabilidad (bootstrap), aplicar a datos reales, probar algoritmos jerárquicos o DBSCAN, y combinar con análisis de valor (LTV).

Reconocimiento

El **reconocimiento** incluye tareas como reconocimiento facial o de voz, donde el sistema identifica patrones complejos. Por ejemplo, en seguridad, el reconocimiento facial permite controlar accesos.

Sistemas de recomendación

Estos sistemas sugieren productos o contenidos basados en el comportamiento del usuario y patrones aprendidos. Netflix recomienda películas y series, mientras que Amazon sugiere productos relacionados.

Introducción a Deep Learning

El **Deep Learning** es un subconjunto de ML que utiliza redes neuronales profundas para modelar funciones complejas y extraer características automáticamente de datos sin procesar. Por ejemplo, en

reconocimiento de imágenes, Deep Learning ha superado métodos tradicionales al identificar objetos con alta precisión.

Nota: Deep Learning requiere grandes cantidades de datos y potencia computacional, pero ha revolucionado áreas como visión por computadora, procesamiento de lenguaje natural y más.

Conexión con conceptos previos: Ya conoces los fundamentos de ML y sus tipos básicos. Ahora, al introducir Deep Learning, ampliamos las herramientas para resolver problemas más complejos en la industria.

En la práctica industrial, cada tipo de problema requiere una solución adaptada. Por ejemplo, un banco que quiere detectar fraudes usará modelos de clasificación entrenados con datos históricos de transacciones.

Una empresa inmobiliaria puede usar regresión para estimar precios de propiedades y asesorar a clientes.

Los equipos de marketing segmentan clientes con clustering para diseñar campañas efectivas. En seguridad, el reconocimiento facial automatiza el control de accesos, mejorando la eficiencia y seguridad.

Plataformas de streaming y comercio electrónico implementan sistemas de recomendación para mejorar la experiencia del usuario y aumentar ventas.

El Deep Learning, por su parte, permite abordar problemas con datos no estructurados como imágenes, audio o texto, donde las técnicas tradicionales de ML tienen limitaciones. Por ejemplo, en diagnóstico médico, redes neuronales profundas analizan imágenes para detectar enfermedades con alta precisión.

Esta unidad te prepara para identificar qué tipo de problema estás enfrentando y qué técnica de ML o Deep Learning es la más adecuada, una habilidad clave para cualquier profesional en IA.